

Ritesh Vangala 2025

Building a Mobile Application

CS3821 - Final Report

Ritesh Vangala

Supervisor: Francisco Ferreira Ruiz



Department of Computer Science
Royal Holloway, University of London

Declaration

This report has been prepared based upon only the work that I have completed. Where other published or unpublished materials are used, citations have been provided.

Word count (not including appendix or references): 14506

Student full name: Naga Rithesh Reddy Vangala

Submission date: 11/4/2025

Abstract

This project focuses on developing a productivity application designed to improve users' time management skills whilst enhancing their motivation levels. Key features implemented include an automated to-do list item tagger powered by a machine learning model, a timer with motion tracking capabilities to ensure focus and an analytics page for time management skills. A leaderboard page—inspired by those seen in competitive multiplayer video games—is used to gamify productivity and enhance motivation levels. Developed in Android Studio with a test-driven software development methodology, the app aims to be a useful and engaging tool to help users reach their goals and build healthy long-term habits.

Contents

1	Introduction	5
1.1	Reasons for creating a productivity application	5
1.2	The Problem	5
1.3	Aims and milestones of the project	5
1.4	Improvements compared to other apps on the market	6
1.5	Survey conducted and how feedback was used to improve the application	7
1.6	Planned development process	8
2	Literature review	8
2.1	Review of literature related to leaderboard feature	9
2.2	Review of literature for the implementation of tags	9
2.3	Review of book regarding app development choices	9
2.4	Review of study regarding use of zero-shot classification	10
2.5	Summary	10
3	Android lifecycle with project examples	10
4	Databases	12
4.1	Explanation of development and hardware choices	12
4.2	Features included in tasks backend	13
4.3	Login and registration backend features	17
4.4	Backend operations of image diary feature	18
4.5	Summary	19
5	Security features	20
6	Machine Learning features background and development	21
6.1	Automated tagging feature and its background	21
6.2	Development of task priority feature	22
7	Image diary feature background	23
8	Software engineering	25
8.1	Requirement elicitation	25
8.2	Reasons for using Kotlin and Python for the project	26
8.3	Using GitLab	26
8.4	Testing	28
8.5	Coding standards	30
8.6	Documentation	32
8.7	Application architecture and patterns	33
9	User interaction with application	36
9.1	Opening sequence pages	36
9.2	Analysis/Home page	37
9.3	Leaderboard page	38
9.4	Tasks page	39
9.5	Timer page	40
9.6	Image diary feature	41

10 Assessment	41
10.1 Professional issues for this project	41
10.1.1 Data privacy and security	41
10.1.2 Potential harmful impacts of leaderboard feature	43
10.2 Evaluation and analysis- what went well and what did not	43
11 Conclusion	45
12 Appendix	45
12.1 Resources used	45
12.2 Link to video	46
12.3 Diary	46
12.4 User manual	48
12.5 Original intended project timeline	48
12.5.1 Term 1	48
12.5.2 Term 2	49
12.6 Privacy policy	49
12.7 Health warning	49

1 Introduction

In this section, an overview of the project is given, describing the core details of the application, the development methods, and the problems it aims to solve. It highlights how the app improves users' productivity and incorporates unique features to help users stay organised and motivated.

1.1 Reasons for creating a productivity application

In the original project briefing, details for what the final application should consist of were revealed. The objectives were to create an application that contains useful features that make use of sensors on Android mobile devices in an interesting way. The application idea selected should also have enough scope and potential to include a variety of distinct features to showcase a range of software and sensor functionalities. Creating a productivity application meant that a broad range of features would be suitable. This maximises the opportunity to make use of sensors and implement complex algorithms and functionalities. This is important as the project specification mentions that innovative, unique features are expected, so an app in this sector can meet these targets.

Developing an Android application improved general knowledge of mobile architecture through hands-on experience with key components such as activities, fragments, and APIs. This helped to attain a deeper understanding of how these components can interact with each other to create a seamless and functional user experience. By integrating these elements into this project, a strong understanding of Android mobile architecture was obtained; therefore achieving one of the key aims of the project.

1.2 The Problem

In this current fast-paced environment, individuals have a variety of responsibilities. Productivity, time, and workload management are all vital skills that are necessary to ensure they can complete important tasks and reach their goals. Having an application to store their plans in an organised manner can provide relief and peace of mind, as they are surrounded by many distractions. Users can have fluctuations in motivation on a day-to-day basis, due to external factors out of their control. This is a problem, as motivation can be a key driving factor pushing them towards their goals.

1.3 Aims and milestones of the project

The application implements an automated tagging feature where tags are automatically assigned to each to-do list item. This is done using the zero-shot classification machine learning approach. A model can detect key terms in task descriptions and assign suitable labels accordingly. For example, if a user types "Go for a run", the "run" keyword is detected and the item is given an exercise tag. The use of tags saves the users' time and allows them to more easily locate items they are looking for, improving their organisation skills [1].

To build upon the machine learning functionality, a task prioritisation feature has also been included. This makes use of zero-shot classification in a unique way. Priority labels are weighted and normalised, to rank tasks based on priority effectively. It also takes into account the completion time for higher accuracy. The process enhances users'

decision-making by guiding them to complete the most important tasks first, ensuring good time management.

The project implements a timer into the application, where users can decide work lengths to optimise their productivity. The feature makes use of built-in proximity sensors to detect movement, indicating when a device has been picked up. Based on this, a notification is sent to the user, to guide them back to concentrating on their work. This feature holds users accountable, ensuring they stick to the tasks they have assigned for themselves. A structured approach to time management is ensured, helping users maintain focus and achieve their goals [2].

The application implements a tasks page to hold users' tasks along with their categories and the number of hours they will spend. All of these details are stored in a database and can be accessed by other features of the app to display information regarding the completed tasks. This creates a cohesive experience, where a variety of features are interlinked to provide the user with as much information as possible. This allows them to understand exactly what they need to prioritise to meet their targets.

The analytics page is one of these connected features, where users are shown a graphical breakdown of how they have spent their time in each category. By viewing these graphs regularly, they can reflect on the progress they have made and refine and optimise their work accordingly.

For user motivation, a leaderboard page has been implemented. The leaderboard tracks the number of hours that each user has worked and converts them into points. When the users see their friends' point levels, it inspires them to compete by gaining more points through completing more tasks. This feature creates a gamified competitive environment, making completing tasks more enjoyable for users.

An image diary feature has also been implemented. The details for it have been described in the survey section.

The application's navigation will be contained in a bottom navigation bar for ease of access. This simplicity is important as if many hierarchies are in place, users may become confused in terms of how to reach their desired features [3].

1.4 Improvements compared to other apps on the market

A key area that distinguishes this application from others on the market is the unique leaderboard feature. Whilst this functionality is commonly found in games and fitness-tracking apps like Strava, it is rarely integrated into productivity applications. The feature is designed to significantly enhance user motivation due to the competitive environment it creates. This feature borrows the socially-driven motivation aspects from fitness apps and brings them to the productivity space, making it unique in helping users keep to their goals long-term.

Another distinctive feature is the automated tagging system. Nearly all other productivity apps allow users to organise tasks with categories, but this application automates this process using machine learning techniques. This saves the user extra time and effort and allows for better task organisation. Few other apps on the market have a feature like this, so users can have access to menus which have been automatically organised and decluttered.

For instance, TickTick, a popular application on the market, offers features including task management and organisation. However, as many tasks are added, manually adding tags can become cumbersome. The automation of this repetitive process allows users to

save time. Additionally, TickTick allows for adding due dates to set reminders but lacks the competitive elements that motivate users to stay on track and achieve their goals. To improve upon this, the leaderboard feature was created.

This application stands out in the productivity space by combining a variety of features from multiple sectors into one. By addressing core pillars of productivity that many other apps overlook, it sets itself apart from traditional task management solutions.

1.5 Survey conducted and how feedback was used to improve the application

Before beginning the development process, several potential users were interviewed and asked a series of questions relating to what they would expect from a productivity-based application. They were queried on what aspects of currently available apps they appreciated and used regularly, their productivity habits and the challenges they face when aiming to manage their time and workload. The feedback provided valuable insights and became a core aspect of development.

One of the questions that was asked regarded the emotional impact that productivity tools can have. One participant stated that, while many current task management apps create a brief bell reward sound upon task completion, this signal of satisfaction was brief and did not provide a lasting sense of accomplishment. The participant had a desire for a feature that would act as an acknowledgement of their achievements, reinforcing motivation and satisfaction for the long term.

To address this need, the image diary feature was created. This feature allows users to visually document their progress by taking photos. Each image is stored automatically and saved along with corresponding task titles as captions, creating a visual timeline of accomplishments. The diary acts as a record that users can revisit regularly, invoking a greater sense of fulfilment than a brief bell sound. The lasting visual reminder of progress allows users to stay engaged. Being able to use the camera for this feature also fulfils one of the project aims to implement the use of sensors into the application to create innovative features. To further justify this inclusion, a study has shown that the practice of taking photos regularly can create a sense of routine [4]. This routine can then expand to cover the continued completion of a variety of tasks and keep users accountable through visual commitment.

Another group of participants also wanted features that enabled them to interact with others, whilst minimising distractions. To meet these needs, the leaderboard feature was created, allowing users to compare their progress with their friends, fostering a competitive, but encouraging environment. By checking their ranking regularly, users are encouraged to stay on track with their tasks. This community-driven element helps users remain focused and inspired. The use of a leaderboard also means that users are not able to directly communicate with each other, preventing unnecessary distractions. This, along with a balance of friendly competitiveness, ensures users can benefit from the motivational aspects of social comparison.

These are examples of how user feedback and needs shaped the overall direction of development, ensuring the app meets functional requirements and also tailoring it to meet the needs of potential users, ensuring users will continue to use the application over a long period.

1.6 Planned development process

The way all of these features will be brought together, is with the use of Android Studio and the programming language Kotlin, as it is versatile and officially supported by Google [5]. Java (a commonly used language) and Kotlin share many similarities, so the learning curve is minimal and there are plenty of resources available to learn from. Android Studio provides an efficient environment to operate in as it includes auto-complete features, imports libraries automatically and offers real-time previews during UI design.

The application will be developed using a TDD (Test-Driven-Development) process, so features operate reliably, leading to a cohesive application [6]. It also improves code quality and developers' confidence in the product being created. Unit testing in particular will be required to ensure specific features are working as intended without bugs before moving on to focus on another feature.

To design the user interface, Figma will be used. It is a simple tool to operate so multiple prototypes can be created to visualise the types of interfaces which will be most suitable. Exporting key assets to Android Studio afterwards is a straightforward procedure, meaning that UI design in XML is more approachable.

All backend features for the application will be hosted on a Python backend. Python is a widely used versatile language that is simple to use and understand, making it ideal for developing backend systems. Many libraries are available such as Flask, transformers and bcrypt, allowing for the development of secure and efficient development of APIs. In particular, Flask's modular structure allows for easy addition or modification of features without the need for significant restructuring.

Version control will be handled using Git, which tracks changes throughout the development process. The software allows for branching and merging, which allows for independent feature development to avoid disrupting the rest of the codebase. It also provides a clear history of modifications, making it easy to identify any errors and reverse any changes if required. This allows for a structured development process and reduces the risk of losing progress.

The rest of the report will describe in detail the background of the features that have been implemented, the app's architecture and the software engineering techniques that have been used. It will also reflect on the project as a whole, and discuss professional issues and potential future improvements.

The next section of this report is the literature review, which discusses the studies and articles that informed the decision to create a productivity app. The review will focus on the effectiveness of features, like the tagging and leaderboard features, to improve productivity. The articles provided important information on how these elements can contribute to a motivating and organised user experience.

2 Literature review

This literature review will discuss articles that led to the decision to create a productivity application. Productivity is universally regarded as important, but this section aims to focus on scientific findings regarding how the application's features allow for a productive environment to be achieved. The review will also discuss articles that guided decisions regarding how to structure the app and the development process.

2.1 Review of literature related to leaderboard feature

One of the key topics of literature had to do with how leaderboards and gamification can aid user motivation (hence increasing their productivity). In a study conducted by Gerdenitscha et al. 114 members who used Habitica, an app that aims to gamify tasks, were studied. Data showed that the gamification process had a positive impact on users' enjoyment of their work and also led to an increase in their productive output [7]. This included many participants stating that the gamified elements triggered them to begin tasks in a timely fashion and avoid procrastination.

Although this study does not follow the long-term impacts (in terms of if extended usage causes "gamification fatigue", where users no longer feel the positive effects), the short-term impacts are very promising, especially for users that struggle with procrastination and maintaining focus.

Another study by Mekler followed 295 users as they were told to categorise images with the use of tags. When finished, based on a user's group, they were shown either their score, level or their leaderboard ranking. Compared to the users who were only shown their score or level, the leaderboard users showed a significant improvement in performance [8].

The two studies are conducted in different areas, but both show positive results, thus indicating the versatility of this type of feature. From conducting this research, it can be concluded the inclusion of a leaderboard feature is justified.

2.2 Review of literature for the implementation of tags

Vuorikari's article titled *Tags and self-organisation: a metadata ecology for learning resources in a multilingual context*, emphasises the use of tags and their value for organisation skills. The results from the article show that participants find that tags are useful for streamlining their workflow and enabling them to focus on specific contexts which are of most importance [1]. For example, tags for specific areas such as "Exercise", "Work" or "Personal" differentiate from each other, and by organising with tags they were able to prioritise based on the category most important to them. This shows the practical uses of such a system, hence why the inclusion of a feature to automate tagging is justified. It takes responsibility for the organisation aspect of the tasks away from the user, allowing them peace of mind and the chance to spend their time in other important areas.

2.3 Review of book regarding app development choices

The book *Creating Dynamic UI with Android Fragments* by Jim Wilson describes the need for fragments in Android applications to supplement existing activities and simplify the creation of user interfaces [9]. Fragments improve over the former standard style (using only activities) by using modular and reusable UI components that can be dynamically added or removed from the activity context. This leads to the creation of flexible and adaptive interfaces, which makes it easier to manage app components and improve the user experience in general.

Managing multiple activities can also make UI navigation complicated as activities tend to load slightly slower than fragments, so holding specific features (other than the login and registration pages) is unsuitable. As described in Chapter 5 of the book, the way intuitive navigation can be created is through the use of easy access buttons.

In particular, the book details the use of an ActionBar, where a specific instance of a fragment is associated with each button in the bar.

The book, however, describes an old-fashioned approach where the icons for navigation are all placed at the top, meaning they can be hard to reach. To improve upon this, the application stores these navigation buttons at the bottom of the screen and uses icons instead of words for a cleaner look. This also brings the application in line with other modern applications, making it easier to interact with.

Chapter 4 of the book describes how to manage all fragment navigation through just one activity for a variety of features including back button support and seamless transitions between features. Due to these benefits, the application holds all navigation to classes in just the HomeActivity, the first page seen after the user logs in.

2.4 Review of study regarding use of zero-shot classification

A study conducted by Yin et al. titled *Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach* provided valuable insight into the zero-shot classification [10]. It details how zero-shot classification allows for associating labels for data without the need for specific training, making it very versatile for real-world applications. This makes it particularly useful for the creation of an automated tagging feature, despite the model not being trained beforehand.

However, whilst it is very adaptable, the performance and accuracy are significantly impacted by the choice of model. Therefore, while the inclusion of this feature is justified, a model trained on a varied dataset is required for optimal operation.

2.5 Summary

Understanding and studying these articles and also the others in this report was crucial to grasp how important applications like these can be to the overall well-being of users. The research also guided project development decision-making, to ensure a good user experience, optimally implementing the features.

Further details on articles researched while creating this report can be found in the bibliography.

The next section of the report will outline the Android activity and fragment lifecycles, particularly methods that handle changes in application state. It also explains the importance of resource management and app optimisation.

3 Android lifecycle with project examples

This application makes use of the Android lifecycle to manage changes of state efficiently, ensuring smooth operation and optimisation.

It is the set of states that an application goes through from creation and destruction. The Android operating system manages these processes to optimise resource usage and ensure applications operate smoothly. For example, the system can unregister listeners for sensors to free up resources and conserve battery life.

When an activity is launched, the onCreate() method is executed to carry out startup operations including setting up the user interface. This is where the layout is defined by linking to an XML file using the setContentView() function. For this application, the

first main process is to handle the user login process before transitioning to the main dashboard.

The next step in the lifecycle is the `onStart()` method, which is responsible for making the activity visible to the user and allowing for interaction. For example, when a user opens a fragment, the method ensures the needed UI components are initialised and ready for use. Immediately after, the `onResume()` method is also called, making the fragment the full focus of the application. Upon navigation away, the `onPause()` method is called, temporarily halting all processes related to the fragment.

The `onResume()` method is called when switching between fragments, ensuring the UI reflects any recent changes that have been made. For example, after ticking off tasks and adding an image diary entry via the alert dialog, `onResume()` refreshes the tasks fragment upon return to the page, ensuring the display is up-to-date and relevant.

```
1  override fun onResume() {  
2      super.onResume()  
3      refreshTasksList()  
4  }
```

When the fragment is opened by the user, the updated tasks are shown to the user.

After a user has finished using a feature and navigates elsewhere, `onStop()` is called. The fragment itself will not be visible, but remains in memory for ease of re-access, without needing a full restart. While a process is stopped, the resources it occupies are released to optimise performance.

If the user plans to return to a specific fragment the `onRestart()` method is called, which then invokes `onStart()` once again to reinitialise components and resume the process.

Fragments in general follow a very similar lifecycle to activities but have some key differences such as requiring a connection to a host activity. In this application, all of the key features are held inside fragments, which are accessed via the `HomeActivity`. The fragments supplement the activity to create a more cohesive navigation experience for the user [9].

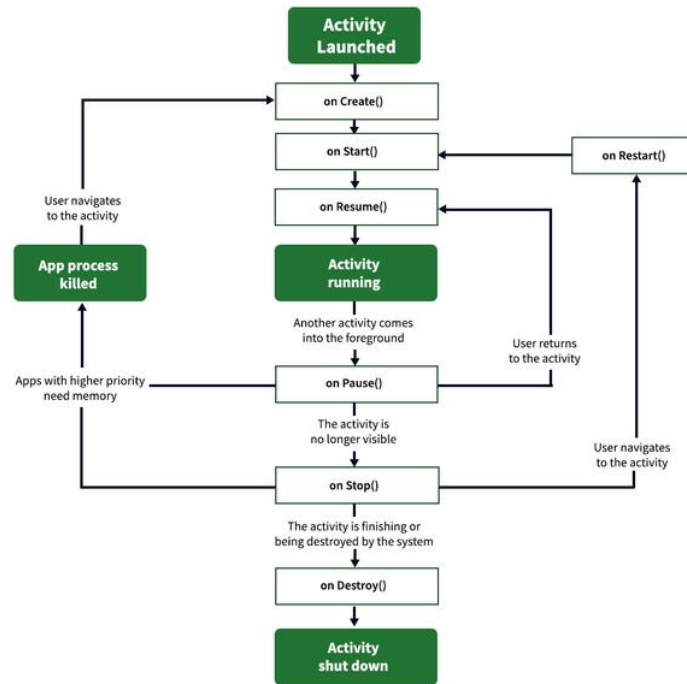
```
1  binding.bottomNav.setOnItemSelectedListener { item ->  
2      when(item.itemId){  
3          R.id.home_nav_button->startHomeFragment()  
4          R.id.timer_nav_button-> startFragment(TimerFragment())  
5          R.id.setting_nav_button -> startFragment(TagsFragment())  
6          R.id.tasks_nav_button -> startTaskFragment()  
7          R.id.leaderboard_nav_button ->  
            ↪ startFragment(LeaderboardFragment())  
8      }  
9      true  
10  
11  }  
12  fragmentManager = supportFragmentManager
```

This snippet shows how all fragments of the application can be accessed through the HomeActivity.

The key functions that are different in fragments in comparison to activities are `onAttach()` and `onDetach()`. The `onAttach()` method is called whenever a fragment becomes associated with an activity. The fragment gains access to the activity's context allowing it to access resources and perform actions.

The `onDetach()` function is similar to that of `onDestroy()` for an activity, where the key goal is to release resources and nullify any listeners or callbacks that are in use.

By using fragments in `HomeActivity`, the application achieves a modular design where user navigation becomes intuitive and resources are efficiently managed.



Activity Lifecycle in Android

Diagram to visualise android lifecycle process.

The next section describes how the backend databases for the application have been created and also how the application can access and update the database when needed. It also includes information on how the database models and functions are defined and the hardware used to power the operations.

4 Databases

4.1 Explanation of development and hardware choices

Databases are a crucial part of this project and are required for many key features such as the login/registration functions and holding users' tasks. Prior knowledge from the CS2855 Databases module was vital to set up the database schema and SQL was needed to query and update the database. Due to prior familiarity with these concepts, the majority of the focus could be directed to the implementation, for which Flask and Flask-SQLAlchemy libraries were used in Python.

All backend functionality is hosted on a Raspberry Pi 5, which is an ideal device to use for its portability, ease of use and affordability whilst having enough computational

power and space to run most of the functions required reliably. The eight gigabytes of RAM present allow the system to run all required databases simultaneously, without experiencing slow response times from any of them.

During the initial phases of development, the database system was stored locally on the user's device, which was effective for storing tasks. However, as login and registration features were implemented, along with leaderboard features to support multiple users, a local option was no longer feasible. Therefore, a dedicated server was required. This also allowed users to log in and view their tasks from multiple devices.

SQLite is the lightweight relational database management system used to store and manage application data. It does not require a separate server process making it ideal for a smaller project like this. It is also well tested- the test scripts have 1084 times more code than the SQLite library itself and 100 percent branch coverage is claimed [11]. This level of testing indicates the level of confidence users can have in its reliability. It is used in web browsers, mobile devices and car infotainment systems. Its ability to handle a variety of situations makes it ideal for this project.

4.2 Features included in tasks backend

```
1 tasks_backend = Flask(__name__)
2
3 tasks_backend.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///tasks_holder.db'
4 tasks_backend.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
5
6 db = SQLAlchemy(tasks_backend)
```

Code shows how simple the database setup is, only requiring one file to hold all information a feature.

The first step taken to create this database was defining a Task model, where each of the task attributes corresponds to a specific column in the database and each row represents an individual task. Establishing a clear structure for the model at the start ensures consistency throughout the development process. The attributes themselves were also carefully chosen to ensure only essential details were stored to maximise efficiency. Each task instance is forced to have a unique ID to ensure integrity and prevent accidental deletions. This structured approach also meant that any potential future modifications would be much easier to implement and integrate.

```
1 class Task(db.Model):
2     id = db.Column(db.Integer, primary_key=True)
3     title = db.Column(db.String(100), nullable=False)
4     tag = db.Column(db.String(25), nullable=False)
5     hours = db.Column(db.Integer, nullable=False)
6     username = db.Column(db.String(25), nullable=False)
7     due_date = db.Column(db.Date, nullable=True)
```

Code shows how the model has been defined.

In terms of functionality and connecting the application to the backend, the ability to add to the database was programmed to begin with. The required details are extracted from data received in POST request forms (lines 3-8). These were then used to create

a new object to insert into a database (lines 17-20) (there are separate databases and objects for different features e.g. task objects for task features and user objects for login features). A message is then sent to be displayed in the Android app via the use of a Toast notification to provide information about the operation and allow the user to know if the task has been added successfully (lines 15 and 22).

```
1 @tasks_backend.route("/add_task",methods=['POST'])
2 def add_task():
3     id = request.form["id"]
4     title = request.form["title"]
5     tag = request.form["tag"]
6     hours = request.form["hours"]
7     username = request.form["username"]
8     str_due_date = request.form.get("due_date")
9
10    due_date=None
11    if str_due_date:
12        try:
13            due_date=datetime.strptime(str_due_date,'%Y-%m-%d')
14        except ValueError:
15            return 'error'
16
17    new_task=
18    ↪ Task(id=id,title=title,tag=tag,hours=hours,username=username,due_date=due_date)
19
20    db.session.add(new_task)
21    db.session.commit()
22
23    return "Successfully added task"
```

Code shows how the function to add tasks operates.

The next step of development was to define the functionality needed in order to remove the tasks upon completion. Since every Task object has a unique ID, this was the way the tasks were queried from the database. Initially, the tasks were distinguished based on their descriptions; however, this in rare scenarios can lead to conflicts where tasks may be deleted unintentionally, so the unique IDs were used instead. As the user ticks off a task in the application, the unique ID of it is sent to the backend, so that the process can begin. When the function is run, the completed tasks are first moved to a separate table in the database (lines 8-16), as some details, such as the hours spent, are needed to provide other important features, including the analysis and leaderboard pages. After this is done, the tasks can then be removed from the database (lines 18-19). Toast messages are displayed to inform the user if the process has been successful (lines 21-23).

```

1  @tasks_backend.route("/delete_task/<int:task_id>", methods=["DELETE"])
2  def delete_task(task_id):
3      task = db.session.get(Task, task_id)
4
5
6      if task:
7
8          completed_task = CompletedTask(
9              id=task.id,
10             title=task.title,
11             tag=task.tag,
12             hours=task.hours,
13             username=task.username
14         )
15
16         db.session.add(completed_task)
17
18         db.session.delete(task)
19         db.session.commit()
20
21         return {'message':f"Task {task_id} deleted"}
22     else:
23         return {"error":"Task not deleted"}

```

Code shows how function to delete task operates.

In the same Python file as the tasks backend, a function to retrieve the total number of hours worked per tag has been implemented, so that the pie chart view can be shown to the user. Whilst this is not a function for the tasks feature, it was suitable to include in the same program, so that the database can be easily queried to obtain the required details. To obtain the values, the completed tasks table is queried and while iterating through, the total hours are summed up and returned in the form of a dictionary, where the tags are keys and the values are the total hours spent (lines 5-11).

```

1  @tasks_backend.route("/get_hours_per_tag")
2  def get_hours_per_tag():
3      tasks = CompletedTask.query.all()
4
5      hours_per_tag = {}
6
7      for task in tasks:
8          if task.tag in hours_per_tag:
9              hours_per_tag[task.tag] += task.hours
10         else:
11             hours_per_tag[task.tag] = task.hours
12
13
14
15     return {"hours_per_tag":hours_per_tag}

```

Code shows how the table is queried and total hours are summed up before returning the dictionary

To be able to process this data in Android Studio, the response is read as a string before being converted into a JSONObject (lines 1-3). The keys from the dictionary are then called to be able to obtain the number of hours worked per tag to display the analysis page (lines 5-19). A similar method is also used to obtain the number of hours worked per user, to provide data for the leaderboard feature.

```
1  val responseBody = response.body?.string()
2  val jsonResponse = JSONObject(responseBody.toString())
3  val hoursPerTag = jsonResponse.getJSONObject("hours_per_tag")
4
5  val workHours = hoursPerTag.optInt("Work",0).toFloat()
6  val personalHours = hoursPerTag.optInt("Personal", 0).toFloat()
7  val exerciseHours = hoursPerTag.optInt("Exercise", 0).toFloat()
8  val shoppingHours = hoursPerTag.optInt("Shopping", 0).toFloat()
9  val uniWorkHours = hoursPerTag.optInt("Uni_Work", 0).toFloat()
10 val gardeningHours = hoursPerTag.optInt("Gardening", 0).toFloat()
11
12 requireActivity().runOnUiThread {
13     updatePieChart(workHours,
14                     personalHours,
15                     exerciseHours,
16                     shoppingHours,
17                     uniWorkHours,
18                     gardeningHours)
19 }
```

Code shows how information is extracted from response to create pie chart

The functionality to display the tasks was created next. When the user switches to the tasks fragment of the application, a GET request is immediately made to the backend to be able to retrieve all the tasks which are relevant to the user who is currently logged in. Once all the data has been fetched, a list of dictionaries is then created, containing each of the relevant fields of a Task object (lines 3-14). The function then returns the list of tasks as a JSON object, which can easily be converted into Task objects, when they are displayed to the user in a RecyclerView. Having already defined a Task object in Android Studio, the integration with the backend is seamless and retrieval times are very short.

Later on during development, a category selection button was added to the tasks page to allow users to retrieve tasks based on their tags. A separate backend function was created to provide this functionality, where the database is queried and only the tasks matching a specific tag are retrieved and displayed. The tag selected is passed through the API endpoint URL so that the Flask server knows exactly what tasks to obtain and display to the user.

```

1  @tasks_backend.route("/get_tasks")
2  def get_tasks():
3      tasks = Task.query.all()
4      task_list = [{
5
6          'id':task.id,
7          'title':task.title,
8          'tag':task.tag,
9          'hours':task.hours,
10         'username':task.username
11     }
12     for task in tasks
13     ]
14
15
16     return {"tasks":task_list}

```

Code shows how function to get tasks operates.

4.3 Login and registration backend features

Although adding a task was relatively simple to define, adding the details of a new user to the database required much more consideration. One of the reasons for this is that many people may choose to select the same username for using the app. However, this may create conflicts in the database, as usernames must be unique to differentiate and correctly identify users from each other. To solve this, the function first queries the database to check if the same username has already been registered. If so, a Toast message is sent to the user, informing them that they must choose another unique username (lines 13-14). Security is another key concern, as the users' passwords must be stored safely in order to prevent potential data theft. This is explained in detail in the security features section.

```

1  @login_register_backend.route('/register', methods=['POST'])
2  def register():
3      username = request.form['username']
4      password = request.form['password']
5
6      user_exists = User.query.filter_by(username=username).first()
7
8      password_bytes = password.encode('utf-8')
9      salt = bcrypt.gensalt(rounds=14)
10
11     hashed_password = bcrypt.hashpw(password_bytes,salt)
12
13     if user_exists:
14         return "Username taken"
15     else:
16         new_user = User(username=username, password=hashed_password)
17
18         db.session.add(new_user)
19         db.session.commit()
20         return "User Added Successfully!"

```

Code shows how function to register users operates.

The function to log users in shares some similarities with the registration function, as user details are extracted from a POST request (lines 3-4). Next, a search of the database is conducted to ensure that the given username is present in the database (line 6). If not, then the user is informed of this via a Toast message, so they can check if they are typing their details correctly. The given password is then checked to see if it matches the one in the database so that they can log in. In a situation where the username is entered correctly but the password is not, a specific Toast message will notify the user of this. By structuring the login features in this way, users will be able to identify and correct any mistakes they are making when typing their details, meaning that the overall process can be completed quickly.

```
1  @login_register_backend.route('/login', methods=['POST'])
2  def login():
3      username = request.form['username']
4      password = request.form['password']
5
6      existing_user = User.query.filter_by(username=username).first()
7
8      password_bytes = password.encode('utf-8')
9
10     check_password_bytes = existing_user.password
11
12
13     if existing_user:
14         if bcrypt.checkpw(password_bytes, check_password_bytes):
15             return "Logged in successfully"
16         else:
17             return "Incorrect username and password combination"
18     else:
19         return "User does not exist"
```

Code shows how function to log users in operates.

4.4 Backend operations of image diary feature

Image objects for the image diary feature are also kept on the backend along with pictures of each image stored in a separate uploads file. The file is a 'static/uploads' folder so that the images can be accessed over HTTP. Uploads to the database are completed using POST requests, where an image and the caption (corresponding to the task title) are sent (lines 3-4). The uploaded images are available to be displayed in the app's RecyclerView using Glide to load the images quickly (lines 21-30). The fast loading times of several images means that the experience is almost as fast as having the apps stored locally, providing a smooth user experience and ensuring they do not notice any delays in image rendering.

```
1  @image_diary_backend.route('/upload', methods=['POST'])
2  def upload():
3      image = request.files['image']
```

```

4     caption = request.form['caption']
5
6     if image:
7         image_name = secure_filename(image.filename)
8         image_path =
9             ↪ os.path.join(image_diary_backend.config['UPLOAD_FOLDER'],image_name)
10
11         image.save(image_path)
12
13         new_image = Image(image=image_name,caption=caption)
14         db.session.add(new_image)
15         db.session.commit()
16
17     return "added image"
18
19
20 @image_diary_backend.route('/get_images', methods=['GET'])
21 def get_images():
22
23     images = Image.query.all()
24     images_list = [{
25
26         'image':f"http://192.168.1.112:4997/static/uploads/{image.image}",
27         'caption':image.caption
28     } for image in images]
29
30     return {"images":images_list}

```

Code shows functions used to upload and obtain images.

4.5 Summary

The OkHttp library was used in Android Studio to connect the backend to the frontend. It allows for HTTP connections to be made, allowing POST, GET and DELETE requests to be sent and their responses to be received. OkHttp allows the data exchange process to be completed rapidly, for fast authentication, task retrieval and task classification. The library's built-in connection pooling (using existing connections for many requests, reducing latency and improving performance) contributes to these fast response times. Asynchronous requests can be made via the OkHttp client to allow for the user interface to operate smoothly and maintain a pleasant user experience.

To summarise, the backend for the project is built upon a strong foundation making use of the Raspberry Pi 5 to host the functionality and SQLite as the database management system. The Flask and Flask-SQLAlchemy libraries in Python have been used to create a seamless connection between the frontend and backend. Key features include user authentication, task management and data analysis capabilities. The OkHttp library is used to facilitate connections between the frontend and backend ensuring fast data exchange. The style of architecture used allows for scalability, security and optimal performance, allowing for opportunities to expand the feature set in future.

The next section discusses the security features that have been implemented to protect

users and the integrity of the database systems. It focuses on cryptographic techniques for password storage along with security measures to prevent common attacks such as denial of service and brute force.

5 Security features

For the login and registration features, security is of high importance to ensure that the users' data can be kept as safe as possible. Prior knowledge from the IY2760 Introduction to Information Security and IY2840 Computer and Network Security modules contained information on how to store data safely. The security implementations in the backend employ some of the security principles and cryptographic techniques taught.

When a user signs up to use the application, the bcrypt hashing algorithm is applied to the password after it has been converted into bytes (must be in bytes to input into the algorithm). A salt is then added to the password before hashing. The salt will be a random and unique value to enhance security and protect against common attacks. One of the attacks prevented is a rainbow table attack, where attackers possess a table containing common passwords and their corresponding hashes; the salt being unique means the hash will be different from the one stored in the database, so an attack will not be possible. This approach also prevents hash collisions, meaning that if two users have the same password, their hashed values will be different and unique, hence increasing security. Hashing also means that plaintext passwords are not stored, so user details are not compromised even if the database is exposed.

As a user logs in, the password is converted to bytes and checked against the hash stored in the database. The function used to compare the two hashes is constant-time, meaning that timing attacks will be prevented, where attackers aim to deduce information about the password based on the time taken for the operation to complete. If the password matches the stored hash, then the user is authenticated and logged in.

Rate limits have also been put in place to restrict the number of requests made from a single IP address. This helps protect against brute force and denial of service attacks. If a user exceeds these limits, then an error will be shown notifying that they have made too many attempts, so the system will not be subject to excessive traffic.

```
1  #used to stop excessive requests and prevent denial of service attacks
2  limiter = Limiter(
3      app=login_register_backend,
4      key_func=get_remote_address,
5      default_limits=["100 per hour", "30 per minute"]
6  )
```

This code shows how limits were placed for all requests in order to prevent attacks.

By incorporating hashing, salting and rate limitations, the system is capable of effectively defending against a variety of cyber threats. The use of hashing means that even if the database is compromised, user details will remain secure. This approach strengthens the overall security of the application, allowing for a secure and reliable experience.

The next section will describe the theory and core functionality behind the automated tagging feature to assign categories to task items. It describes zero-shot classification the core logic behind the feature, the Python library and the model used.

6 Machine Learning features background and development

This section discusses the two machine learning features present in the application. It outlines the theory behind how they operate and the thought process behind their implementations.

6.1 Automated tagging feature and its background

The application includes a feature which automatically assigns a tag to a task item upon entry. The functionality is powered by zero-shot classification, which is a technique that enables the system to categorise text without any prior training on each specific category. A variety of different topics can be input and adapted to, meaning that this form of machine learning is suitable for this feature [10].

The approach is based on transfer learning, where a model generalises knowledge from previously learnt data to apply it to new unseen categories. A key difference between zero-shot classification and other transfer learning methods is that it does not require any prior fine-tuning before being used for a new purpose. Zero-shot classification aims for generalisation to classify a variety of inputs with minimal prior knowledge.

To bridge the gap between known and unknown categories, semantic embeddings are used. This is where data maps are formed in a semantic space containing attributes and keywords, to allow the model to infer relations and classify data into new categories.

When classifying a task, the model takes an entailment approach and finds the percentage chance that a given input belongs to a certain category by evaluating the relationships between the input and potential labels. This method allows the model to adapt to a variety of tasks without interruptions, providing an efficient classification method.

The Hugging-Face transformers library in Python was used for this implementation, providing access to a wide range of pre-trained models and classification techniques. The model chosen in particular, was the *facebook/bart-large-mnli* model. As it was trained on the MultiNLI dataset, the model is very versatile and capable of handling a range of different topics without additional tuning. The model processes the input task title alongside an array of possible labels, returning a list of potential labels in order of likelihood. For this use case, the output was adapted to return only the most probable label.

After the system had been set up and tested with a variety of sample inputs, it was embedded into a Flask API so that the Android app could connect and access these features. The communication was facilitated via the OkHttp library.

```
1  #function to classify text
2  @tags_classifier.route("/classify",methods=['POST'])
3  def classify():
4      #Gets text input
5      todo_item = request.form["value"]
6
7
8      #Conduct classification
9      result = cf(todo_item, candidate_labels)
10     label = result['labels'][0]
11
12     #Returns result
```

```

13     print(label)
14
15     #Result obtained by the app
16     return label

```

Snippet of Python code shows how the application uses the input from the application to conduct the classification, and returns the most probable label.

This implementation provides a fast and seamless tagging feature, which helps improve user experience by automating an organisational process with minimal latency. By eliminating the manual tagging process, the feature enhances overall productivity and organisation, making the app more intuitive.

6.2 Development of task priority feature

When developing the task prioritisation feature, the initial intention was to use a feature outside the Hugging-Face library. This was to ensure a diversified feature set to be able to use a variety of different algorithms and techniques. This approach would allow for an opportunity to experiment with a variety of machine learning models, which help to make a specific tailor-made feature for this process.

However, despite this, a suitable large dataset containing to-list items and priority levels, could not be found. A dataset with a large varied sample set would have been required for this feature to train a supervised model effectively, to make it capable of prioritising tasks. Creating individual samples and labelling them, would have been time-consuming and given the short time frame available, this was not feasible.

Due to these constraints, the implementation had to pivot and go back to using zero-shot classification, as it did not require a training dataset to be operational. For it to fit this use case, the scores for each label were obtained and assigned a specific weight. The average of all these was then calculated to give the final score. Assigning priority values in this way is appropriate, as it takes into account all the probabilities, where the highest probability labels are given the most weight. The outcome will be a fair, accurate score.

```

1     score+= result['scores'][0] * score_values[result['labels'][0]] * 1
2     score+= result['scores'][1] * score_values[result['labels'][1]] * 0.6
3     score+= result['scores'][2] * score_values[result['labels'][2]] * 0.3
4     score+= result['scores'][3] * score_values[result['labels'][3]] * 0.1
5
6
7     score = score/2 #To normalise the scores (divide by sum of weightages)
8
9
10    #High importance tags mean higher score
11    if tag == "Work" or tag == "Uni Work":
12        score *= 2
13
14    #More routine tags mean lower score
15    if tag == "Gardening" or tag == "Personal":
16        score = score/2
17
18    #Rest of scores remain the same

```


The code above shows how scores were weighted based on likelihood and how the category selected impacted the final priority score.

This current formula acts as a strong base, which can be fitted once a larger, high-quality dataset becomes available. A potential way of collecting data (if consent from users is granted) could be to gradually build a dataset of task descriptions and associated priority ratings. When enough information has been collected, a supervised model can be used to learn from the data, so that an accurate and personalised prioritisation system can be implemented.

Overall, this feature showcases how machine learning can be utilised to formulate strategies for optimal workflow and completion. By having the order in which to proceed pre-made for users, they will avoid decision fatigue, which in many cases can be detrimental to overall output. It ensures that users complete tasks in an order that aligns with their aims and deadlines.

The next section of this report is an in-depth explanation of the image diary feature.

7 Image diary feature background

This feature has briefly been mentioned in the databases section. Here, more detail is provided.

This feature integrates a Flask backend to the application to store images taken by users' mobile devices along with their captions to view them in a RecyclerView for easy scrolling. The OkHttp library is used to facilitate communications and ensure fast loading speeds.

The /upload endpoint accepts an image and its caption. It uses the *secure_filename()* function to check through the filename to ensure that the data being uploaded does not include any harmful characters (lines 3-4). For example, certain special characters or particular sequences can be uploaded in a way such that access to forbidden sections could be granted, so this function is run to replace or remove these characters. If a filename was to be written in a specific way, then a malicious user could potentially have access to other users' data, so this security feature is of high importance. Once this process is completed, the names are stored securely to protect against attacks.

```
1 @image_diary_backend.route('/upload', methods=['POST'])
2 def upload():
3     image = request.files['image']
4     caption = request.form['caption']
5
6     if image:
7         image_name = secure_filename(image.filename)
8         image_path =
9             ↪ os.path.join(image_diary_backend.config['UPLOAD_FOLDER'], image_name)
10
11         image.save(image_path)
12
13         new_image = Image(image=image_name, caption=caption)
14         db.session.add(new_image)
15         db.session.commit()
16
17     return "added image"
```


This snippet shows how images and captions are uploaded to the backend and how the files are stored with secure path names.

The `/get.images` endpoint provides JSON responses that contain the image URLs and their captions so that they can be processed by the application and displayed to the user. In the application, the `parse.images()` function is used to store details in `ImageItem` objects. A list of these objects is shown to the user via a `RecyclerView`. This entire process, despite its complexity, occurs almost immediately after accessing the fragment.

```
1  @image_diary_backend.route('/get_images', methods=['GET'])
2  def get_images():
3      images = Image.query.all()
4      images_list = [{
5
6          'image':f"http://192.168.1.112:4997/static/uploads/{image.image}",
7
8          'caption':image.caption
9      } for image in images]
10
11     return {"images":images_list}
```

Snippet shows how images are retrieved from the database.

The code below demonstrates how a picture taken is then prepared for upload. The bitmap is compressed into a JPEG format, before being converted to a byte array. In this particular instance, a `MultipartBody` had to be used instead of the usual `RequestBody` as a wide variety of types need to be sent in the request to ensure that the image object can be formed. The images are then stored in a `static/uploads` folder which means that they can be obtained and displayed via their unique URLs. The `Glide` library is used to load the images into the `RecyclerView`.

```

1      val randomNumber = Random.nextLong(10000)
2
3      // The bitmap is transformed to a byte sequence so that it can be uploaded
4      ↳ to the backend
5      val stream = ByteArrayOutputStream()
6      ↳ bitmap.compress(Bitmap.CompressFormat.JPEG, 100, stream)
7      val byteArray = stream.toByteArray()
8
9      // A request body is created so that only jpegs can be uploaded to the
10     ↳ database, for security reasons
11     val requestBody =
12     ↳ byteArray.toRequestBody("image/jpeg".toMediaTypeOrNull())
13
14     /**
15     * A multipart form is needed for this situation as a variety of types
16     * ↳ need
17     * to be uploaded in one request.
18     *
19     * An image, caption and username are sent to the backend to be stored
20     * as an image diary object.
21     */
22     val multipartBody =
23     MultipartBody
24     .Builder()
25     .setType(MultipartBody.FORM)
26     .addFormDataPart("image", "task-image\${randomNumber}.jpeg",
27     ↳ requestBody)
28     .addFormDataPart("caption", taskTitle)
29     .addFormDataPart("username", usernameText)
30     .build()

```

During the early development and testing process a *'/display'* endpoint and a basic HTML site were used to check that the images and the captions were being passed correctly.

This feature ensures a structured and efficient manner of storing user-generated images, whilst ensuring security and integrity. Security is particularly important in this case, as the pictures stored likely will be deeply personal so potential leaks would reduce users' trust in the app.

8 Software engineering

This section of the report will cover the software engineering aspects of the project, including how features were tested, version control usage and maintaining good code quality. It will also explain and justify programming design and language choices.

8.1 Requirement elicitation

To gather information to decide the requirements for the project, potential customers were surveyed (as mentioned prior). Their feedback was vital so that their expectations

and habits could be understood. This helped shape core features such as the image diary feature.

Review and testing of other applications on the market was useful to understand what features competitors provided that were effective and popular amongst users. The process revealed gaps and limitations with existing apps, which led to the inclusion of unique additions to cover some key areas that have been neglected.

By combining effective elements of existing apps along with new ones created using feedback, the final set of requirements was comprehensive and set a strong guide to follow during the development process. The process also allowed for the creation of an app that has a combination of familiar and new features, making it more approachable for users.

8.2 Reasons for using Kotlin and Python for the project

This project is developed primarily using Kotlin for the main application implementation. One of the primary reasons for this is that it is officially endorsed by Google, meaning that there is plenty of documentation and tutorials to learn from. Kotlin is very similar to Java, and as Java is widely used for a variety of projects and assignments, it meant that learning to use Kotlin was a simple process.

To further aid the learning process, the number of Kotlin-based queries has been growing exponentially since the official Google endorsement, meaning that there is access to a large, active community that helps answer queries [12]. Previously asked questions will also aid the development process as many problems encountered during development will have already been addressed, making the solutions easy to access.

On top of this, Kotlin has many advantages over using Java, including concise syntax, better null safety systems and it is easier to read. All these advantages significantly reduce the chance of errors. Kotlin also supports coroutines for asynchronous programming. As the application's core features make calls to a backend, this allows for efficient handling, ensuring UI updates remain smooth while waiting for server responses. This improves the overall performance and user experience.

Despite these pros, Kotlin is fully interoperable with Java, meaning members of development teams can select the language they are more comfortable with, without introducing any compatibility issues. This level of flexibility is valuable for larger teams, resulting in adaptable and productive work environments.

The combination of null safety mechanisms, developer resources and easy to understand syntax makes Kotlin a perfect choice for Android development.

To create the backend server for the application, Python was used due to its ease of use and the wide variety of libraries available. It has a simple and easy to read syntax, which helps to create modular, maintainable applications. The lightweight Flask web framework was used to create the APIs and define their endpoint functions. It allowed for easy integration with databases and user authentication, due to the variety of libraries used in combination. Flask's SQLAlchemy extension streamlined database operations, making it easier to set up and query tables.

8.3 Using GitLab

GitLab served as the version control system for application development, providing a structured environment for code development. By making use of features such as branching and commit tracking, the development process followed a clear path, which allowed

for the application's feature set to be expanded systematically.

Separate branches are used to develop each major app feature. This allowed multiple aspects of the app to be developed independently and concurrently without affecting the main codebase. For example, the development of the tasks fragment section did not affect the rest of the app. This isolation of features also presented opportunities to add depth to specific areas, without being burdened by the needs of another. This process is how the image diary feature came about, as a way of adding an extra layer to the user experience to meet the goals of the project. Therefore, using multiple branches creates a more free environment which can give developers confidence to experiment with and test new ideas, hence leading to innovation.

When specific features had been completed and finalised, they were then merged back into the feature branch they were created from. This helped to ensure that the main branches were all kept up to date with the latest updates, ensuring that new features can be integrated into the rest of the application smoothly. This process was only done once a feature had been fully tested (physically and with unit tests) so that conflicts would be minimised. Due to these steps, the overall project was clean and organised. After merging was completed successfully, some older branches were deleted, to avoid unneeded clutter, so that only active branches remained. As the development process began to conclude, most of the feature branches that were a key part of development were merged together and eventually merged into the main branch to mark the finalisation of development.

During the early stages of development, a practice branch was also created to experiment with and understand Android Studio's tools, Kotlin and UI components. A separate space to improve development skills at the start of the project was important to understand the IDE and general Kotlin conventions.

Given that most of the app's features depended on each other to operate - needing to share variables or objects - a series of feature branches were created. When a feature that relied on an existing one was to be developed, a branch was created off it. For example, the registration feature branch was created from the opening sequence branch, to ensure users are directed to their desired pages. This method lessens the risk of integration issues and speeds up the development process so that key functions and variables don't need to be repeatedly rewritten. The taskFiltering branch was also created from the taskDueDate branch, as the new feature required the use of existing functionalities, such as displaying tasks and adding them. To test the filter, adding tasks was required, so forming the branch in this manner meant necessary features were available without the need for writing additional code for setup.

Commits to update the diary or feature branches were done frequently to ensure clear traceability. This frequency and the detailed commit messages made progress tracking simple and made rollbacks straightforward when necessary. This disciplined approach helped maintain a clear development history. Having commit descriptions that were well written also helped to fix errors that arose during development and to look back on code to be more well-informed before developing a new feature.

The diary file was also updated often to log progress, identify potential roadblocks and gauge the time commitment required for the development of a new feature. Due to the limited development time available, this was a suitable way to ensure the most impactful and time-consuming features were completed first before moving on to others.

Overall the use of GitLab throughout allowed for a systematic development approach with the use of branches, commits and other useful features. Independent branches minimised intrusions, and frequent commits allowed for traceability and easy rollbacks. The

diary file, which was updated frequently, helped to track progress, manage dependencies and prioritise key features.

8.4 Testing

During the development process, testing was conducted throughout to confirm that key aspects of the application operated as intended and were able to handle the variety of ways that users will interact with them. A mixture of unit tests and instrumented tests were used in order to ensure isolated aspects of the app worked effectively. For some situations, such as testing interaction with servers and databases, instrumented tests proved to be unsuitable, leading to unit tests being used instead. Using both testing methods in combination, ensured validation of core app logic, whilst also confirming real-world functionality is as intended.

To test the user interface, the Espresso library was used. Espresso is a framework used to make writing UI tests efficient and reliable. It provided a framework to simulate the actions that real users take when using the app such as pressing buttons, ensuring accurate text is displayed to read (e.g. the correct greeting based on the time of day) and that the correct fragments and activities are launched at appropriate times. A key feature of Espresso is that it automatically syncs with the UI thread, which means additional wait commands do not need to be written. Additionally, Espresso seamlessly integrates with JUnit, ensuring structured and maintainable tests are written.

The Espresso testing framework was used to ensure proper functionality of the navigation buttons within the HomeActivity. The test verifies that each button, when clicked, directs the user to the correct corresponding fragment and that the navigation system is operating as intended. By simulating button clicks, the test ensures that the buttons are displayed and clickable. It also ensured that after navigating to another fragment, the buttons remain visible, so that the user can navigate to other sections of the app. The syncing capabilities of the Espresso library ensure that tests run reliably and make the navigation process during testing run smoothly. Overall, this test helps to ensure this core app logic is functional. Also, as the Espresso library simulates real user interactions, it therefore helps to test how intuitive a user interface is.

```
1      @Test
2      public void testNavigation() {
3          onView(withId(R.id.home_nav_button)).perform(click());
4          onView(withId(R.id.home_nav_button)).check(matches(isDisplayed()));
5
6          onView(withId(R.id.tasks_nav_button)).perform(click());
7          onView(withId(R.id.tasks_nav_button)).check(matches(isDisplayed()));
8
9          onView(withId(R.id.timer_nav_button)).perform(click());
10         onView(withId(R.id.timer_nav_button)).check(matches(isDisplayed()));
11
12         onView(withId(R.id.setting_nav_button)).perform(click());
13         onView(withId(R.id.setting_nav_button)).check(matches(isDisplayed()));
14     }
```

This test shows button presses are simulated and the display is checked to ensure that the correct fragment is displayed to the user.

Key variables and objects were mocked so that the functionalities of the application could be tested thoroughly. For example, a calendar object was used and its time was set specifically, so that it could be adapted for each greeting. This ensured that all possibilities were tested.

```
1      @Test
2      public void testMorningGreeting() {
3          calendar.set(Calendar.HOUR_OF_DAY,9);
4
5          String greeting = homeFragment.setGreeting(calendar);
6
7          assertEquals("Good Morning!",greeting);
8
9      }
```

This test sets the time of the calendar object before testing to check that the message reflects the time of day.

In order to test the database retrieval functionality, mock objects were used to simulate the API functionality and check if information is retrieved and processed correctly. The MockWebServer library from OkHttp was used to initialise a mock backend server and also a mock client in the setup() function of the test class. These functions are ones that are run before all tests to ensure the server and client are both initialised beforehand to ensure a controlled test environment for reliable results. A request was made to the mock server and the response was converted to a Task object via Gson. Assertions were then made to ensure that the details from the response had been extracted and passed correctly to a Task object. Once this process has been completed, the teardown() function is called to create a clean test environment for next time. This system is effective as it allows for fast-to-run and repeatable tests that isolate specific functionalities.

```

1      @Test
2      fun testJSONResponse() {
3
4          val mockJsonResponse = """{
5              "id": 99,
6              "title": "Go for a run",
7              "tag": "Work",
8              "hours": 5,
9              "username": "Jon"
10         }"""
11
12         mockWebServer.enqueue(MockResponse().setBody(mockJsonResponse))
13
14         val request = Request.Builder().url(mockWebServer.url("/task")).build()
15
16         val response: Response = mockClient.newCall(request).execute()
17
18         val task = gson.fromJson(response.body?.string(), Task::class.java)
19
20         assertEquals(99, task.id)
21         assertEquals("Go for a run", task.title)
22         assertEquals("Work", task.tag)
23         assertEquals(5, task.hours)
24         assertEquals("Jon", task.username)
25     }

```

This code is used to test that a response from the server is correctly processed and that the object parameters store the correct values.

An example of where testing was able to find crucial errors in the code was in the analysis page feature. During testing, it was uncovered that one of the sectors of the pie chart was not being updated with the values from the backend. In order to fix this error, the source code responsible for handling this was examined and it was discovered that there was a typo in the 'Uni Work' tag where an extra space was added, which meant that the required data could not be queried from the database.

This issue had not been caught during initial development or through UI observation, highlighting the need for systematic testing. It was through these tests - designed to verify that the chart accurately reflected the data - that the bug was discovered. Without testing, this subtle issue may have gone unnoticed and misled users by providing inaccurate data.

8.5 Coding standards

Maintaining high coding standards is vital for ensuring good code quality, readability and consistency across all classes over the course of development. To enforce a standardised code style, Ktlint and Detekt plug-ins were used in the project. Ktlint was used to apply consistent formatting by following a standard style guide, automatically correcting issues like improper indentations and line spacing. Another plug-in Detekt, was used to identify potential code smells and gave suggested best practices for code. It flagged issues like magic numbers, long lines of code, unused imports and broad imports along with warnings about exceptions that are not specific enough for their situations.

Magic numbers, which are hardcoded values without meaningful variable names, were recommended to be replaced by static variables to allow for readability and maintainability.

```
1 return when (hourOfDay) {
2     in 0..11 -> "Good Morning!"
3     in 12..17 -> "Good Afternoon!"
4     else -> "Good Evening!"
5 }
```

This shows how difficult to understand this section of code can be.

This section was therefore updated to ensure user readability.

```
1 return when (hourOfDay) {
2     in 0..MORNING_END -> "Good Morning!"
3     in (MORNING_END+1)..AFTERNOON_END -> "Good Afternoon!"
4     else -> "Good Evening!"
5 }
```

Replacing magic numbers with variables makes the code clear, easy to understand and more maintainable. Changes that need to be made can be done so from a single point rather than being redefined across multiple areas in the codebase.

Broad imports can clutter the code and introduce dependencies that would otherwise not be required. The compilation times of code can also be significantly increased, which wastes time and computational resources.

```
1 import okhttp3.*
```

This broad import can increase compile time and include unnecessary dependencies.

Below is the improved method of importing libraries.

```
1 import okhttp3.Call
2 import okhttp3.Callback
3 import okhttp3.OkHttpClient
4 import okhttp3.Request
5 import okhttp3.Response
```

Specific imports improve code clarity, enhance maintainability and speed up compile times.

Using tools like Ktlint and Detekt ensured consistent code formatting and identified issues like magic numbers and broad imports. They played a key role in enforcing consistency, improving readability and streamlining the development process. Following these practices led to higher quality code and made it easier to manage the project over the long run.

8.6 Documentation

Clear and detailed documentation is important to maintain high code quality and standards. It allows other developers to understand the contexts and purposes of functions, variables and general architecture. This allows them to build upon the existing codebase and implement new features, therefore creating a collaborative environment. Additionally, errors due to misunderstandings will be minimised and navigating through code will be much easier [13].

In this project, KDoc was used extensively throughout to provide structured comments for functions, their parameters, fragments and classes. It ensures that the logic behind core functionalities can be easily interpreted by all members of the development team. The approach taken included high-level documentation used to generally describe fragments and functions and inline comments to explain specific algorithms, like loops and calls to the backend. The inclusion of high-level comments in particular allows programmers to navigate through the codebase quickly as they only need to read to first few lines, to understand a file.

```
1  /**
2   * This fragment displays users' tasks
3   *
4   * The users can add and delete tasks
5   *
6   * Can filter tasks by tag
7   *
8   * When ticking off a task, they can take a picture of a task they have
9     ↪ completed
10  */
11  class TasksFragment : Fragment() {...}
```

This section of code shows the purposes of fragments were described clearly and succinctly.

The documentation, as shown above, provides a clear and succinct overview of the purpose and functionalities of the fragment. Each functionality is explained on separate lines, making them easy to understand and makes the comments more readable.

```
1  /**
2   * Task is classified and the correct category is selected automatically
3     ↪ for the user
4   *
5   * @param taskTitle The data to be classified
6   * @param spinner the spinner from which the tag is selected
7   *
8   * Once classified the tag is automatically selected from the spinner
9   */
10  private fun classifyTask(
11      taskTitle: String,
12      spinner: Spinner,
13  ) {...}
```

This snippet shows an example of a function that takes in two parameters and the operation of the function is described.

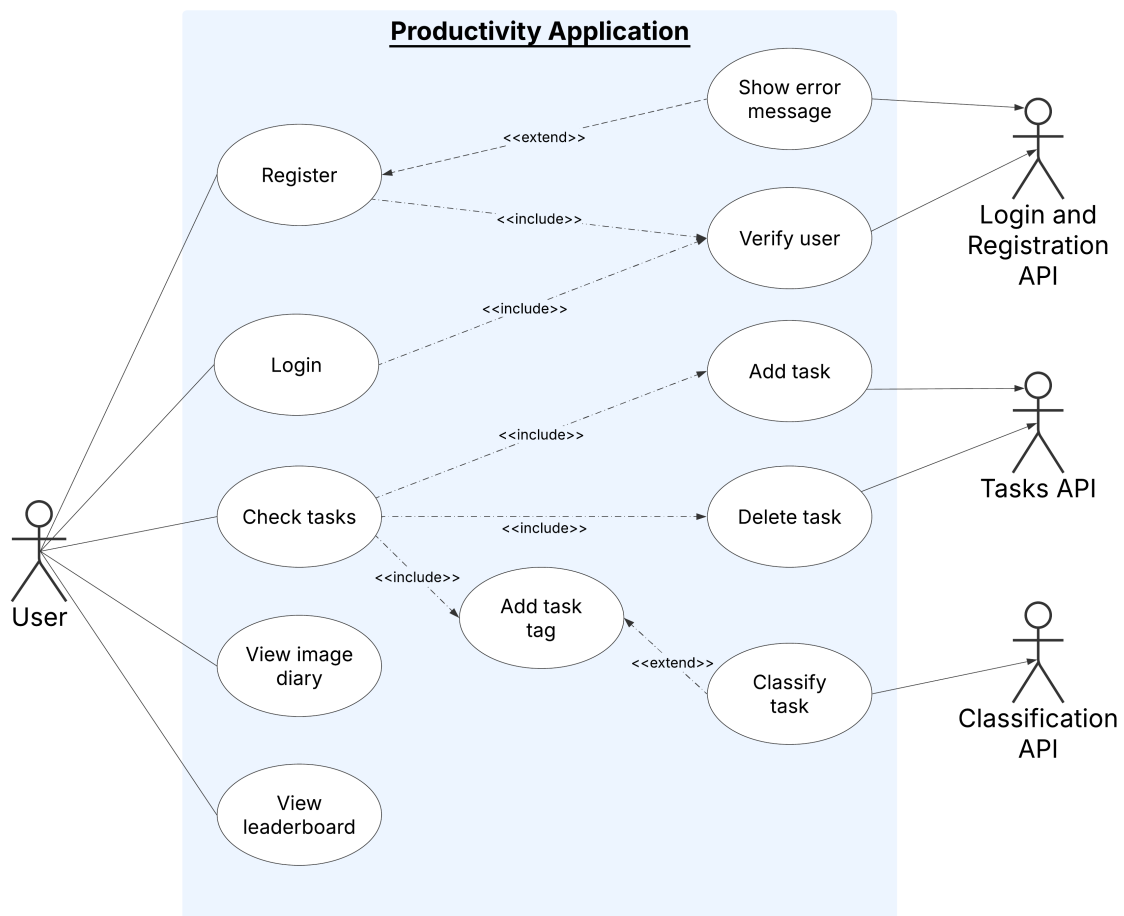
In situations where functions do not return any values, their actions are clearly described. This was important for functions that updated the UI or switched fragments, as these changes may not be immediately obvious in the code. The function above is an example of this.

Inline comments were also used throughout to explain more specific parts of the code, such as loops and situations where complex actions are executed (including making calls to the backend). They can help to break down larger, complex functions into smaller partitions that are easier to understand.

In the Python backend, comments were used to achieve similar purposes to the KDoc comments. In particular, multi-line comments were used throughout to describe functions, their parameters and their outputs. This ensures that developers can quickly understand the logic behind operations.

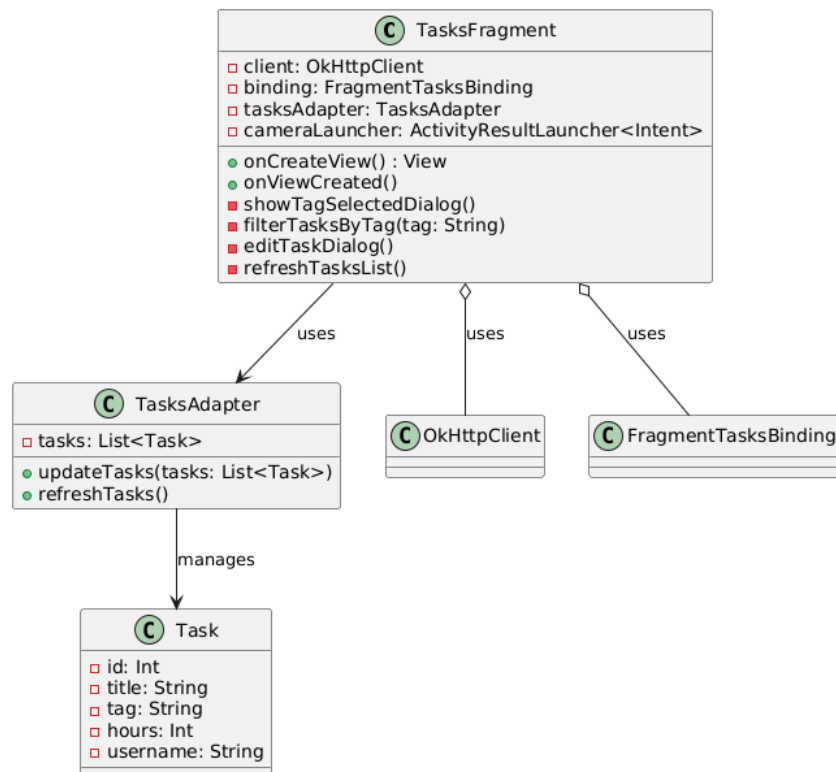
Outside of just providing code explanations, the use of documentation ensures the code remains accessible for new developers joining the team and acts as a form of communication between team members. By providing details with regards to variables and processes, the chance of errors due to misunderstandings is minimal, creating a platform from which the project can evolve.

8.7 Application architecture and patterns



The above image illustrates the key functionalities of the application involving the user and the APIs as the actors. The user can perform a variety of actions including login,

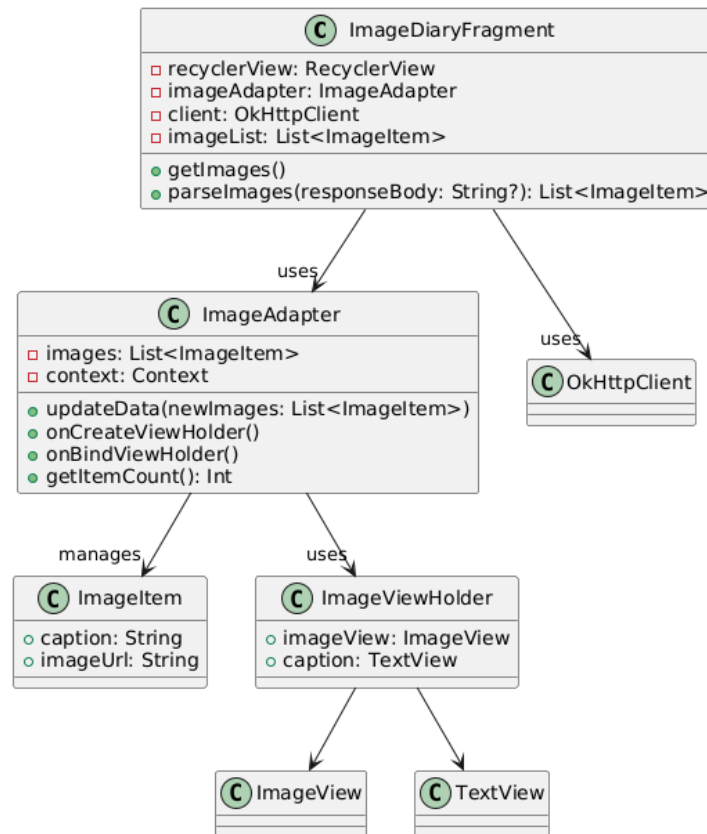
registration and adding tasks. The use cases are connected with the use of <<include>> and <<extend>> relationships, which indicate optional or included functionalities. For example, the login use case includes an additional process of displaying an error message in case login authentication fails, while adding tasks includes a classification feature that automatically assigns tags, provided by the classification API. These relationships show the paths the user can take and also how some features share similarities. For example, both login and registration features rely on similar methods to search the database and verify user credentials, indicating they share common backend operations. Sharing this similarity means that code can therefore be repurposed which promotes reusability and makes the app easier to maintain and scale.



UML diagram shows relationships between tasks fragment, the task data class and task adapter.

The architecture above shows how different aspects of the task feature have been separated. The **TasksAdapter** controls how data is presented. The **OkHttpClient** handles backend communication. A **Task** object is also used to hold core data. By separating these, changes to one part of the application are far less likely to affect the others. For example, the **TasksAdapter** could be modified to change the UI elements, and these changes would be seamlessly reflected in the fragment. The modularity makes the code easy to understand, test and improve upon over time.

The use of an adapter design pattern means that loose coupling is achieved as the responsibilities of various components have been separated. This approach makes the app maintainable and also ensures flexibility. All these components are independent, meaning that modifications to one, will not cause major unintended effects on the others.



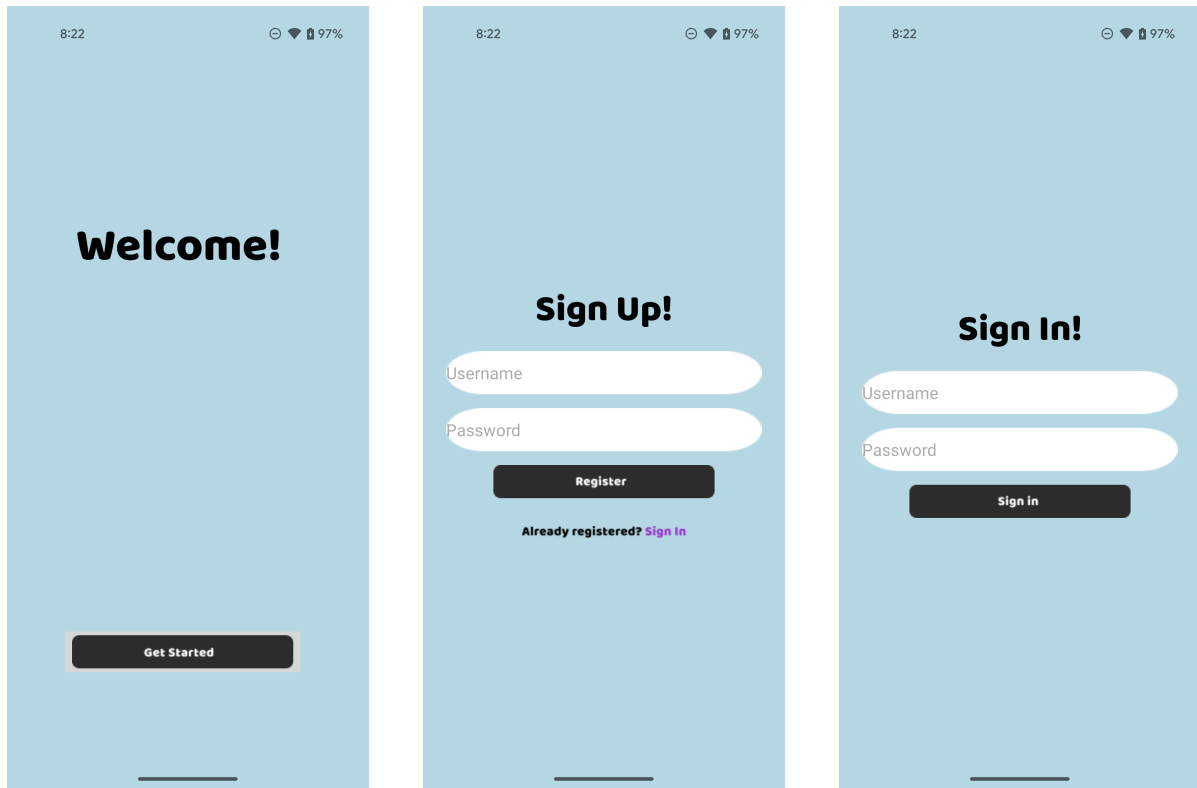
UML diagram shows relationships between image diary fragment, and the image adapter.

The above UML diagram shows how the components of the image diary feature are integrated. It displays a similar structure to that of the tasks feature indicating the versatility of the pattern. The **ImageAdapter** (similarly to the task adapter) serves as a connection point between the database and the fragment, efficiently displaying images and their captions. This adapter also follows the Adapter Pattern, adding data to the **RecyclerView** for fast updates.

In order to create these UML diagrams, PlantUML was used, which is a tool that can help to create diagrams through text descriptions. The tool provided a simple way of inputting class details and connections for them to be displayed visually.

9 User interaction with application

9.1 Opening sequence pages



First page the user sees

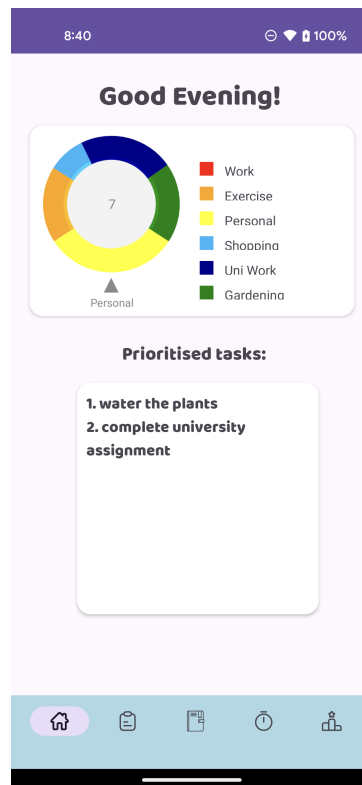
Sign up page

Login page

The images above illustrate the initial pages the user sees upon opening the application. A consistent light colour scheme is used throughout, with a focus on soft blue tones. Light blue is a colour that is associated with calmness and trustworthiness, creating a more inviting user experience that promotes regular engagement [14].

Both the login and sign-up pages are connected to the Python backend, making POST requests for verification and to add user details to the database. This process is completed quickly, so users can access the app without delay.

9.2 Analysis/Home page



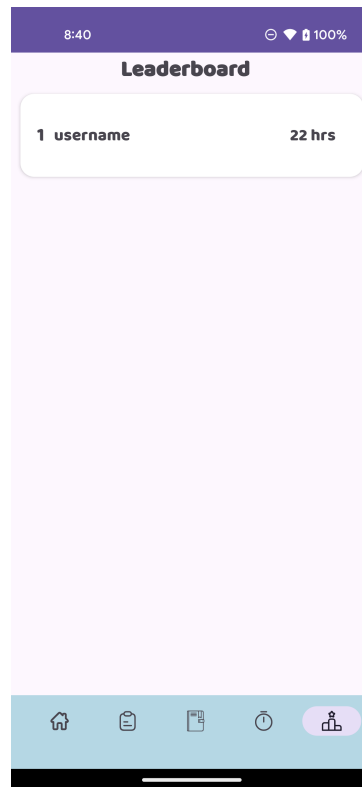
First feature the user sees after login

The image above shows the first feature that the user views upon login. It contains a personalised greeting along with an analysis view that illustrates how the user has allocated their time. Opening with a friendly greeting immediately puts the user in a better mood, so when reviewing the tasks to complete, they will do so with a positive outlook [15].

Having the pie chart immediately underneath emphasises the importance of time management. Users will carry out tasks with an understanding of how they have been spending their time previously, so they can prioritise task categories effectively. By seeing how much time they have dedicated to each sector, they can make informed decisions about how to adjust their schedules and ensure balance. All the different slices are distinctly different colours meaning that colour-blind users can identify separate sectors. As the developer of the application is red/green colour-blind, their personal experiences informed these decisions ensuring accessibility for those with similar visual impairments. The distinct colours mean that over time, users will be able to automatically associate each colour with the corresponding tag, allowing for fast recognition and being able to gauge the time distribution in a short amount of time.

The combination of the greeting and the pie chart improves user engagement and encourages realistic goal setting. It promotes self-understanding and gives users better control of their routines.

9.3 Leaderboard page

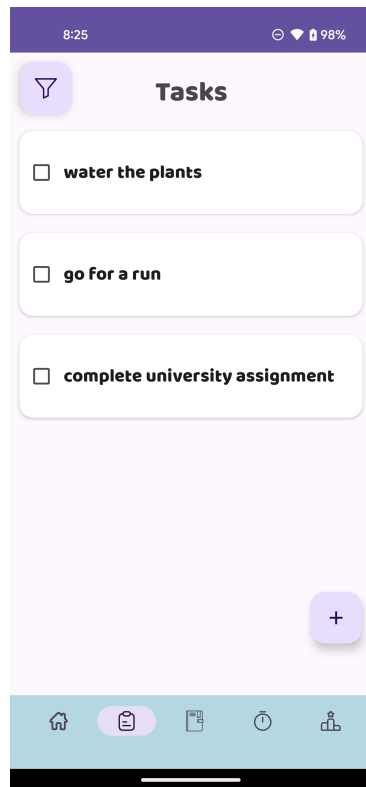


Leaderboard page

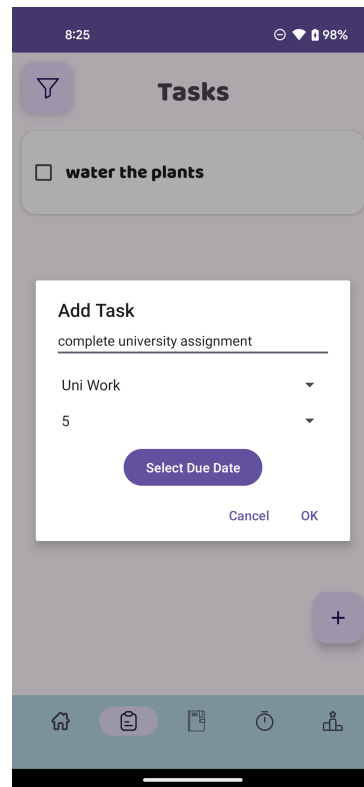
The images above showcase the leaderboard feature, which displays the username along with their associated hours worked. This feature encourages users to compare their progress with their peers and motivates them to stay productive to reach higher positions, promoting motivation and productivity [8]. The leaderboard updates dynamically as tasks are completed, encouraging consistent engagement.

Using this gamified approach builds a sense of community and accountability between users and makes goal tracking a more enjoyable process.

9.4 Tasks page



Tasks page



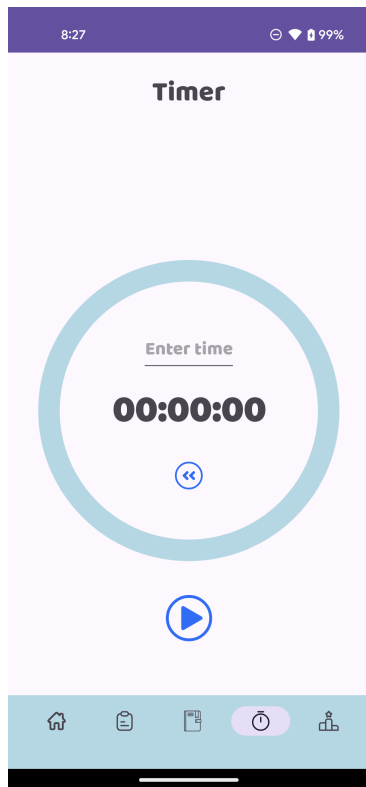
Alert dialog box

The images above display the tasks page and the operations that can be performed. The page serves as a central hub for managing to-do items and displaying them in an organised manner. To add tasks, an alert dialog box is shown, to allow for attribute input, used to add task objects to the database. This intuitive process streamlines task management and reduces the effort needed to stay organised.

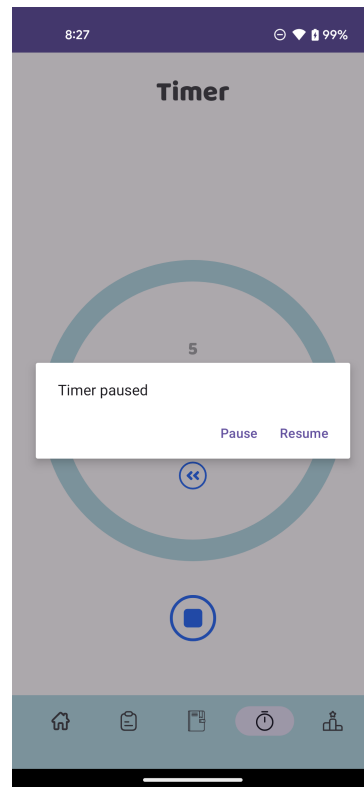
Additionally, as the user ticks off a task, another alert dialog is displayed immediately after, prompting the user to decide if they would to create an image diary entry. If they choose to do so, they are seamlessly redirected to the camera, to allow them to capture an image of the task they have just completed. This is paired with the task title as the caption to be stored in the image diary. The intention behind this design choice create a positive feedback loop by providing immediate satisfaction through visual documentation.

The users can also filter their tasks by tag. As they click the filter icon on the top left corner, an alert dialog with a spinner allows the user to select a specific category. This feature pairs perfectly with the analysis page, where a user may have noticed one of the categories is lagging behind the rest, so they can prioritise it to catch up.

9.5 Timer page



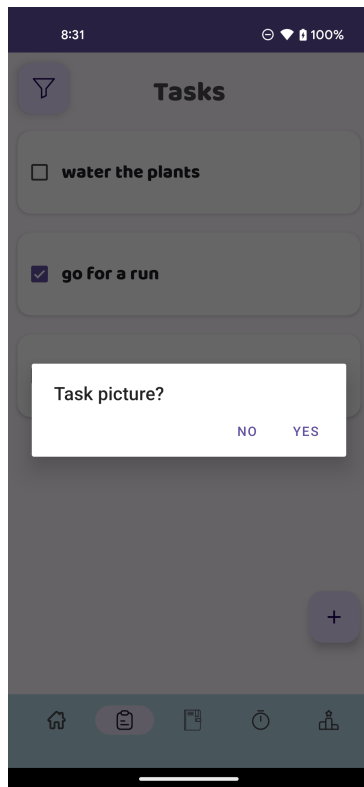
Timer page



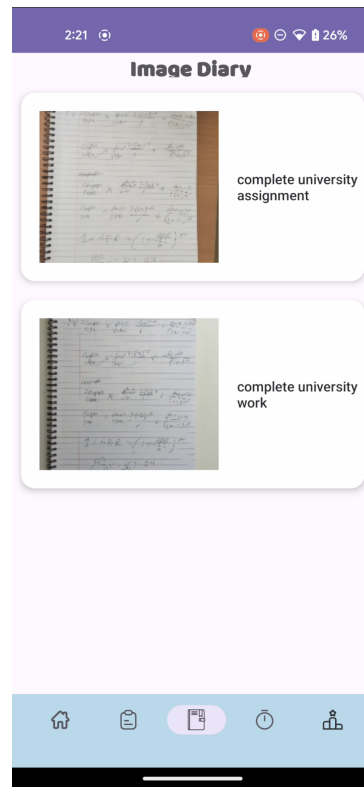
Timer page alert pop-up

This timer page features all essential features for general operation alongside a proximity sensor feature to automatically pause the timer when movement or a nearby object is detected. Another use is where the user can hover their hand over the timer in order to pause it. This hands-free functionality is convenient for users, allowing them to stay focused on their work by not needing to physically interact with their phone. Studies have shown that context switching during work sessions is detrimental to productivity as users report it takes them close to ten minutes to return to a workflow state. A feature like this will save users' time and protect their concentration levels [16].

9.6 Image diary feature



Alert dialog asks user if they would like to add image



Images are displayed alongside captions

The image on the left is the alert dialog shown when a user checks off a task. It prompts the user to decide if they would like to create an image diary entry of the work completed. If they choose to do so, a camera will be launched, allowing them to take a picture which will then automatically be uploaded. On the right is a view of the feature itself, showing the images are neatly displayed, with captions.

10 Assessment

10.1 Professional issues for this project

This section discusses the professional issues surrounding the application. It will cover topics with regards to legal context and user wellbeing.

10.1.1 Data privacy and security

The application collects sensitive information from users' including their passwords, task details, tags, the amount of time they spend per category and their image diaries. It is crucial to handle and store this data securely and responsibly to maintain user privacy and trust.

Whilst this data is only stored to provide personalised experiences for users, it is important to be transparent with how data is processed and who has access to it. To

address these issues, an easy to access and understand privacy policy must be created to detail these practices.

The users' passwords are the most important details to keep safe, as potential compromise can lead to their accounts on this application and others being accessed by unauthorised parties. To prevent this, the passwords are hashed with a leading industry standard algorithm (*bcrypt*) before being stored, meaning that users' passwords will only be known to them and not any adversaries that gain access to the database. For further strength, passwords are also salted prior to being hashed to add an extra layer of security. This therefore prevents rainbow table attacks, where attackers apply hashed versions of commonly used passwords, to gain access.

In order to ensure the images stored in the database are secure, the `secure_filename()` command is used on all file names that are to be uploaded, to ensure no malicious characters are present. These characters can be used to traverse the database and search through the files present. Additionally, as SQLite is the database engine being used, it can be prone to SQL injection attacks, and this function also helps protect against this. Due to a randomising function present in the application code, every filename sent to the backend is unique, meaning overwrites of data will not take place. All these features in combination ensure that user data is kept safe.

Data minimisation should also be adhered to, where only data that is required for operation is collected from users. Collecting additional information can increase the impact that a potential data breach can have. This approach ensures that users maintain trust in the application's security and will continue to use it.

It is also important to comply with all relevant data privacy and protection regulations, such as the GDPR. This includes implementing all required technical measures as required by regulations. All of the core principles of GDPR can be easily met by this application. Lawfulness, transparency and fairness can be met by informing the users how their data is being used. The purpose limitation requirement is met as only data needed to power application features is collected, which therefore also meets the data minimisation requirement. Integrity and confidentiality are also maintained through the use of hashing. It is very important to meet these requirements, as the punishment for failing to do so is €20 million euros or 4 percent of turnover, so the consequences are severe [17].

Following the guidelines directly impacts long-term app success as it helps foster user trust. Complying with the regulations helps build a secure application for growth and user retention, allowing it to thrive in a competitive market.

In general, access control policies and control lists should be applied throughout. These ensure that only those with high-privilege access in the company can handle database operations. Having this is important, as third parties or contractors, who may not have a strong connection to the company, may not be trustworthy, so it is important to restrict their access.

A data breach could have severe consequences, including legal action, financial losses and damage to the company's reputation. This can lead to less engagement from users and regulatory fines. To prevent this from happening, the steps mentioned above have been taken. For further protection, regular audits, policy reviews and real-time threat detection can all be implemented to further align with regulations and industry standards.

10.1.2 Potential harmful impacts of leaderboard feature

The leaderboard feature is designed to be used as a form of friendly competitiveness through gamification. However, this can also potentially lead to unhealthy competition between users. For example, if a user happens to be far behind another on their tasks, this can cause them to become anxious and stressed about their productivity leading them to overwork in order to catch up. This pressure can lead to burnout, decreased motivation and a decline in productivity all together. In addition to this, overworking can lead to detrimental impacts on physical and mental health [18].

Studies have shown there is a strong link between an increase in work time and the risk of developing cardiovascular disease [19]. Prolonged periods of overworking can lead to higher blood pressure, fluctuating sleep patterns and a higher risk of developing anxiety. It is important for users to understand that while work is important, their health must always be prioritised.

Another issue is that users may prioritise quantity over quality and focus on tasks that allow them to attain the most leaderboard points, rather than ones that progress towards their objectives. This goes against a core purpose of the application, which is to build productive habits. To reduce the likelihood of this, lower priority scores are given to these tasks, to push users to complete more difficult challenges first.

To mitigate these risks, users can access the analysis page or the image diary, which provides views of the progress that they have made over time. This allows them to focus on personal development rather than direct competition, reinforcing the fact that the application is intended to motivate and encourage users, rather than placing pressure on them.

Generally, the leaderboard is designed to enhance motivation, but without finding a balance it can lead to stress and unhealthy competition. By implementing a system to scale points based on task importance, the app will be fair and push users to make meaningful progress. The main aim of the leaderboard is to provide encouragement and not pressure.

10.2 Evaluation and analysis- what went well and what did not

Overall, the development process of the application went very well, as nearly all initial software goals set out have been achieved. In cases where the original goals could not be met, they were replaced with similar features that still met the goals. For example, a social media section was originally proposed, but was later replaced by the leaderboard instead. This switch was a suitable choice as the leaderboard fulfilled similar goals to the social page (to motivate and inspire users), whilst also minimising the potential distractions that a social page would introduce. After evaluating the trade-off between social interaction and productivity, the decision was taken to prioritise the latter, to more closely align with core project aims.

Also, as surveys were conducted during the development process, the feedback was taken on board, which resulted in ideas for new features. One of these was the image diary feature, which was a feature that required a large amount of development time including research and manual testing. This was worth it however, as the feature blended seamlessly into the final application. It was easy to use and the upload and retrieval times were practically instant. Using the task titles saves users time from having to re-enter the caption (the task title is usually the perfect caption so manual entry is not required). This allowed users to focus more on task completion, rather than administrative tasks.

A key area for improvement in future is in the backend infrastructure for the task tagging and prioritisation features. Whilst the code implementation was efficient and worked well, the server would have greatly benefitted from increased computational power. The Raspberry Pi 5, despite powering multiple other operations simultaneously (such as login and registration), had limited resources. This constraint meant that wait times for the classification processes could be lengthy, making the feature less convenient to use. Upgrading to a more powerful server or even leveraging cloud-based solutions would improve performance, especially as the number of users grows. A potential cloud-based solution could be Amazon SageMaker along with AWS Lambda. This is a serverless, cost-effective solution which can scale based on the amount of usage, minimising overall operational costs. SageMaker can model training and deployment, while Lambda can handle the prediction requests without the need for managing servers [20].

Alternatively, the Coral TPU (a USB plug-in tensor processing unit by Google) can be used to significantly accelerate the classification speeds. Designed to run with minimal power, these devices pair perfectly with Raspberry Pi's by handling the heavy lifting of classification tasks, boosting performance without placing stress on the main processor. The device was ultimately not used due its high cost, which exceeded the project budget.

A crucial part of the development process that needed careful consideration, was the way multiple features depended on the same sets of data. The leaderboard, tasks and analysis pages all depended on consistently accessing data from the same tables. To ensure seamless integration, the database schema and functions were carefully considered. For example, to ensure simplicity in the backend, the leaderboard page directly obtained details from the completed tasks table to generate user rankings. Since all tasks were associated with usernames, the need for additional tables was eliminated. Adding this parameter ensured that user progress variables were all obtained from a single source resulting in consistency across all dependent features.

Another important consideration was the implementation of the tags classifier feature. Originally, the plan was to include this feature as a separate fragment in the application. However, user testing revealed that this approach offered minimal benefit, as users felt that manually selecting a tag was usually faster than navigating to a separate fragment to conduct classification. As a result, the app design was modified, and for the final application, the classification is conducted automatically after the user has stopped typing for one and a half seconds (to allow time to finish editing their task). The output from the classification is then automatically selected from the tag spinner. This approach improved usability and streamlined the tagging process.

Completing this project also allowed for significant personal growth. The need to obtain and apply a wide variety of skills such as database creation, implementing machine learning features and understanding how to connect frontend and backend operations. It utilised and built upon concepts learnt over the past three years of study. Time-management and decision making skills have also improved, due to the consistency and speed at which milestones had to be achieved. Overall the result of the project, is a functioning application and a strong foundation for further growth in future.

Overall, the project was a success as almost all of the original goals were met and the key features delivered are ones that align with user needs, based on surveys conducted and wider research. Despite challenges during development, effective solutions were implemented, such as delaying backend calls to avoid overloading the Pi 5 server. In future, if the selection of suggested improvements are implemented the app could scale to support a large user base and include machine learning features with faster performance. This

would not only maximise user experience, but also give the app a competitive edge over the market competitors.

11 Conclusion

In conclusion, the project achieves its core goals of providing a variety of features to allow users to manage their workload and maintain positive work habits to reach their goals.

The features implemented build upon existing applications on the market, and aim to combine unique aspects from a variety of sectors, bringing in useful additions that will help in the productivity space. For example, the use of a leaderboard (an idea frequently seen in games and exercise apps) paired with the image diary, addresses competition and self reflection, which in combination encourage personal growth. These features also address the need for external motivation and accountability, as stated in the introduction.

The project also adhered to software engineering standards seen in industry including the use of branches for modular development and consistent use of a version control system to focus on frequent atomic changes. The testing procedures involved unit and instrumented tests, along with giving a device with the application running to potential users. By aligning with the best practices, a quality and functional final product was ensured.

Sensors were also utilised to develop the image diary and timer features, a decision which aligns with the original project criteria provided at the start of the year. The project also meets the demand for innovative features to be included.

Overall, the final application provides a comprehensive user experience to help manage tasks and improve productivity. Technical features like the automated tagging feature, offer intelligent functionality in an easy to access form. The app not only enhances productivity, but it is also practical for daily use.

12 Appendix

12.1 Resources used

Android Lifecycle diagram:

<https://www.geeksforgeeks.org/activity-lifecycle-in-android-with-demo-app/>

Filter icon: https://www.flaticon.com/free-icon/filter_107799

Diary icon: https://www.flaticon.com/free-icon/diary_4388193

Notebook icon: https://pngtree.com/freepng/notebook-icon_8993869.html

Font used: <https://font.download/font/baloo-bhaijaan>

Other icons used:

<https://icon-icons.com/icon/play-blue-button/227848>,

<https://icon-icons.com/icon/stop-blue-button/227854>

<https://icon-icons.com/icon/back-blue-button/227866>

Figma plug-ins used:

Material design kit, Stratis line icons

12.2 Link to video

https://www.youtube.com/watch?v=EjVun08Y3ko&ab_channel=r

12.3 Diary

6/10/24: Today I did some research on Kotlin and the Android Life Cycle. This was an important step as I have decided that this is the environment that I will create my application in. Learning about the language and the Lifecycle will help me during development.

Completed some sketches to try and get an idea of how the opening page of the app may look. Explored ways of moving from tasks page to the social page, either by swiping or adding a dedicated button to the bottom left of screen.

9/10/24: I have received feedback for my project plan draft. I have begun implementing all the advice I have been given.

12/10/24: I have now completed and submitted my final version of my plan, with all the feedback given to me by my supervisor. Now I will begin to follow the timeline that I have created and will begin work on the early deliverables, alongside the timeline. I feel that the early deliverables will be straightforward to complete, so I will dedicate the majority of my time to completing the timeline, starting with creating sketches for how I intend the app to look, and implementing the first features for basic app functions.

15/10/24: I have been learning to use Kotlin and have tested some basic functions on an Android phone that I have found to use for the project. Today I plan to complete some of the early deliverables and will upload the Project Plan.

19/10/24: Completed a basic Kotlin application to get a basic level understanding of how it works. I have been reading articles and watching videos about how to make the UI as suitable as possible. I have looked at the Material 3 design kit which seems to be a fantastic resource. For the AI feature I will look into spaCy and other pre-trained models to see if any are suitable.

25/10/24: Completed splash screen code for application, will now concentrate on creating a logo with notebook and camera, to perfectly capture the scope of the application, and conduct research as to how an app logo can help the user experience.

26/10/24: Implemented a login page. This was important as it will be one of the first pages that the user sees.

28/10/24: Figma work and interim report work. Figma was used to create a mock-up of how the app will look. The intention is to recreate Figma style directly to the XML for real app implementation by exporting required assets to Android Studio. This will help simplify the development process.

2/11/24: Implementing a timer page currently. Focused on completing the general look of the page. Will fix progress bar issues and reset button later, as the first step is to ensure the feature looks good enough for usage.

3/11/24: Completed Figma diagram to understand and visualise flow of application.

Colour was decided for scientific reasons (which included how inviting it has been shown to be for users). Will soon begin to export to Android Studio and add functions. Will then focus on report and learning and creating an AI model to tag items. Also will look into sleep analysis features in order to boost productivity and check if they are suitable for my app.

9/11/24: Bottom nav bar has been created. The features will be made using fragments going forwards. Fragments make changing features much simpler than changing activities and provide a smoother experience for the user. The implementation in fragments is almost the same as using activities, so very little additional learning is necessary.

11/11/24: Hugging Face used for AI tags feature. I chose Hugging Face as it has a variety of models to choose from and is relatively simple to implement in Python after some setup. Key issues are that some tags are very similar (leading less than precise classifications) so more specific tags can choose for more reliable predictions.

12/11/24: Fragment switching implemented, for reasons explained earlier.

13/11/24: Hugging face implemented in flask, timer page drop-down implemented.

15-22/11/24: Database work, including tasks database along with task features. Improved the UI by implementing assets from Figma.

23/11/24: Tags feature with flask backend has been implemented into the app properly. This is an important feature that took some time to get working. Operates while the Flask backend is running.

25/11/24: Proximity sensor feature implemented. Useful feature to easily pause timer without user getting distracted.

28/11/24: Completed supervisor meeting and will act on advice given.

29/11/24: Will look into creating a leaderboard feature if time permits. I have considered creating this feature as an alternative to the social features I proposed earlier.

2/12/24: Completed leaderboard feature. Good feature to implement as it helps with motivation, which is one of the main goals of this project.

25/1/25: Setup raspberry pi to use as server for application to hold tasks, login details and AI feature. Needed feature as currently the login details are being kept on the phone itself which is not suitable

1/2/25: Added code to create servers in Flask, need to add security features for login page and core code will be completed and ready for Kotlin implementation.

5/2/25 - 10/2/25: Updated Kotlin code to work with new backend (login register features)

12/2/2025: Strengthened security and protected against potential brute force attacks by updating flask code for login/register backend

15/2/2025: Updated tasks code to work with new tasks backend.

16/2/25: Used google offered tool to come up with ideas to make backend look more appealing.

20/2/25: Added functionality to store completed tasks in separate table.

24/2/25: Added backend functionality to store images next need to integrate with application.

26/2/25: Can now upload images from app to backend, created html website temporarily to show if images were uploaded to backend correctly (they were). Now need to make these images show in app, caption functionality not working yet need to update or allow user to type custom caption.

3/3/25-6/3/25: App now loads images from backend with captions. Captions now work correctly. Tags feature now implemented to alert dialog with prior errors fixed. Some

timer feature issues fixed still not quite usable yet. Now focus on standout AI powered feature.

8/3/25-13/3/25: Added functionality to get tasks by order of difficulty and fixed various errors throughout application. Added functionality to allow users to include due dates when adding tasks.

14/3/25: Added functionality to score tasks based on difficulty (still being tested).

20/3/25: Updated the scoring system for the priority feature

25/3/25: Tested the priority feature and fully implemented it

30/3/25: Checked codebase thoroughly and added comments and fixed any style errors
Code is complete bar minor things, will shift focus almost completely to the report

12.4 User manual

To run the application, first ensure that all Python backend files are running, otherwise core functionalities will not operate. A virtual environment may need to be setup depending on the operating system used.

To do this run the command:

```
python<version> -m venv <name of environment>
```

Ensure that command is run in same directory the python files are stored.

Double check that once files are run, the required databases are created and the folder to hold images is present. This process should be completed automatically by the Python scripts.

Once this is done, ensure that the BASE_URL variable in the URLs class is set to match your server's address to ensure communications can be made.

Please note that the machine learning features' performance will vary based on the processing power of the server chosen.

Also ensure that camera permissions are granted to use the image diary feature. Upon first time use image may not be uploaded until permissions are given.

Note: Ensure that before going on to use rest of the features in the app, ensure that the task prioritisation call completes fully. This will help to avoid potential crashes.

12.5 Original intended project timeline

Below is the original plan for developing the project. In practice, the development path diverged in many areas including changes to features implemented.

12.5.1 Term 1

Week 1 and 2: Research and finalise which programming language and environment to use for the project.

Week 3: Create various mock pages for how the app may look. Evaluate each one with reference to user psychology.

Week 4: Implement a to-do list page and timer with motion sense features.

Week 5: Begin work on the social side. Implement login and sign up pages.

Week 6 and 7: Begin work on the database to hold user credentials. Research datasets in order to train ML model to make tags for list items

Week 8: Research and evaluate methods of analysing user work hours and production.

Week 9: Add review page to add hours. Implement an analytics page with a feedback system.

Week 10 and 11: Complete interim report and prepare for presentation (External milestone)

12.5.2 Term 2

Week 1: Review progress from previous term. Complete any tasks previously remaining and conduct surveys for feedback.

Week 2: Research and evaluate accessibility options for users

Week 3: Implement most suitable accessibility methods

Week 4: Add features for users to upload and post photos. Include comment and like features

Week 5: Optimise machine learning model

Week 6: : Implement tags feature. Include features to allow analysis based on specific categories.

Week 7: Enhance user experience with animations, smooth transitions and sound effects. Fix any bugs that may occur.

Week 8-9: Evaluate project results

Week 10-11: Focus on final report.

12.6 Privacy policy

The privacy and protection of users' data is of utmost importance. This application only collects data that is required to provide the user experience and no data is shared with or sold to any third-parties. All data is stored securely in encrypted servers. Users' locations are never tracked.

12.7 Health warning

Please ensure that when using the leaderboard feature it is only for the purposes of motivation and personal growth. Hypercompetitiveness and overworking can lead to burnout and cause significant health issues such as cardiovascular disease and anxiety.

References

- [1] Riina H Vuorikari. Tags and self-organisation: a metadata ecology for learning resources in a multilingual context. 2009.

This study was important to my project as it justifies the inclusion of the tagging feature as they have been shown to aid with organisation. The paper explains how tagging can be very helpful and aid users in finding the content they are looking for.

- [2] Awwad J Dababneh, Naomi Swanson, and Richard L Shell. Impact of added rest breaks on the productivity and well being of workers. *Ergonomics*, 44(2):164–174, 2001.

This study shows that giving workers improves overall productivity output, so using this I have decided to include break times with my time. The study follows a test that

was conducted in real life detailing the effects on productivity as workers were given breaks. The study's results showed that short infrequent breaks were not preferred, so my application will only recommend users to select longer times when setting the timer. .

- [3] Qiuyuan Chen, Chunyang Chen, Safwat Hassan, Zhengchang Xing, Xin Xia, and Ahmed E Hassan. How should i improve the ui of my app? a study of user reviews of popular apps in the google play. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 30(3):1–38, 2021.

In this study it is shown that including too many hierarchies to a single app would confuse the users. Due to this I aim to include only up to two hierarchies in my project. The paper's findings also show that app makers interacting with users about their experiences and taking their advice on board makes a positive impact on the app's UI.

- [4] Liz Brewster and Andrew M Cox. The daily digital practice as a form of self-care: Using photography for everyday well-being. *Health*, 23(6):621–638, 2019.

- [5] Marcin Moskala and Igor Wojda. *Android Development with Kotlin*. Packt Publishing Ltd, 2017.

This book explains that Kotlin is a great language to use as it is versatile, easy to read and understand, along with benefitting from support from Google. Therefore I have decided to use this programming language. It is also published by a well known company specialising in learning content. .

- [6] Ayush Vijaywargi and Uchinta Kumar Boddapati. Embracing test-driven development (tdd) in modern android applications.

This is a journal issue that details the variety of benefits of TDD and how it becomes useful for app development. The journal is written by two people which shows it is well written and it contains many references to other articles written by experts in the field. .

- [7] Cornelia Gerdenitsch, David Sellitsch, Markus Besser, Sophia Burger, Christine Stegmann, Manfred Tscheligi, and Simone Kriglstein. Work gamification: Effects on enjoyment, productivity and the role of leadership. *Electronic Commerce Research and Applications*, 43:100994, 2020.

- [8] Elisa D Mekler, Florian Brühlmann, Klaus Opwis, and Alexandre N Tuch. Do points, levels and leaderboards harm intrinsic motivation? an empirical analysis of common gamification elements. In *Proceedings of the First International Conference on gameful design, research, and applications*, pages 66–73, 2013.

This article shows how the inclusion of gamified elements affect users' behaviour. The study shows that using these types of elements can draw users more towards a product, making them more likely to continue using it over a long period of time. .

- [9] Jim Wilson. *Creating Dynamic UI with Android Fragments*. Packt Publishing Ltd, 2013.

This is a book published by a well known company specialising in learning content. It explains in detail how fragments can be used to make the user interface more flexible and discusses the relations between fragment lifecycles and their host activities. It was a vital part of my learning to understand how fragments work. .

- [10] Wenpeng Yin, Jamaal Hay, and Dan Roth. Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach. *arXiv preprint arXiv:1909.00161*, 2019.
This article contains information about how zero-shot learning and how accurate specific implementations are. There was also information about the dataset the model used was trained on. The article was co-written by three authors hence ensuring the accuracy of the data shown and provided deep knowledge of zero-shot learning. .
- [11] Sunny Kumar Aditya and Vikash Kumar Karn. *Android sQLite essentials*. Packt Publishing, 2014.
This book provided information on implementing databases. The information provided was easy to understand and implement. Being able to understand the architecture of SQL was interesting and helpful to design a suitable schema for each use case. There were many practical examples provided which were used to generate ideas for the project. Due to this, the queries written were optimal to ensure speed and efficiency. The book is also published by a well known company specialising in learning content. Whilst, this book described how to use SQL in an Android context, it was useful when creating the python implementation. .
- [12] Victor Oliveira, Leopoldo Teixeira, and Felipe Ebert. On the adoption of kotlin on android development: A triangulation study. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 206–216. IEEE, 2020.
This article provides large amounts of research and information related to the use of Kotlin for Android Development. The particular section that aided this report, is one that provided details on the queries asked on Stack Overflow.
- [13] Diomidis Spinellis. Code documentation. *IEEE software*, 27(4):18–19, 2010.
The article emphasises how good coding documentation involves writing comments that explain algorithms clearly, rather than only restating the code in words. It also highlights that consistency adding code throughout projects is important.
- [14] B West and J Silberman. A colorful impact: the psychological impact of colors. *J. Student Res*, 8(2), 2020.
This paper offers analysis of research conducted by professionals, indicating there are connections between emotions and colours. In particular for this application, the information about a room which was painted blue was useful. Students that were surveyed, indicated that they felt a sense of calmness and serenity, which is the exact same emotions this application aims to instill.
- [15] Haoyu Zhang and Zhuowei Wang. Digital greetings for digital communication. *International Journal of Educational Technology and Artificial Intelligence*, 2(1):1–7, 2023.
This article contains information on how adding greetings to an application can be beneficial for the user experience. Showing expressions in technological environments are known to be appreciated by users, and hence the greeting’s inclusion in the project is justified as it will help their mood and hence their productivity. .
- [16] Qatalog. Qatalog, cornell university’s ellis idea lab (2021) workgeist report ‘21: Research into culture, mindset and productivity for the modern work era., 2021.

The article is from a large team of researchers. They conducted surveys of 1000 participants (the large sample size means their findings provide significant insights into user behaviour) and reported their findings. Particularly important to my application was their findings in terms of context switching where they discovered that a significant amount of time is lost when users switch contexts. This therefore justified the inclusion of the proximity sensor feature.

Accessed: 11 December 2024.

- [17] European Parliament Council of the European Union. Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing directive 95/46/ec (general data protection regulation) (text with eea relevance), May 2016.
- [18] Bhairav Prasad and Charu Thakur. Chronic overworking: Cause extremely negative impact on health and quality of life. *International Journal of Advanced Microbiology and Health Research*, 3(1):11–15, 2019.
This article provides in-depth information and studies regarding how overworking can have serious negative impacts. The link to serious mental and physical health issues is clear, with significant impacts on happiness and fulfillment. As the application includes a potential feature that could lead to this, reading this article was important to understand ways in which the user can alleviate these effects.
- [19] Allard E Dembe, J Bianca Erickson, Rachel G Delbos, and Steven M Banks. The impact of overtime and long work hours on occupational injuries and illnesses: new evidence from the united states. *Occupational and environmental medicine*, 62(9):588–597, 2005.
- [20] Abhishek Mishra. *Machine learning in the AWS cloud: Add intelligence to applications with Amazon Sagemaker and Amazon Rekognition*. John Wiley & Sons, 2019.
This book provides information on the variety of different ways in which AWS offerings can be used to implement machine learning features. It provided in-depth knowledge on how different features can be converged to be able to power the machine learning feature for the application. From these findings, two suitable options were selected for a potential way to enhance the feature in future.